Московский государственный технический университет
им. Н. Э. Баумана

Проект СЭЛФ

МОДЕЛЬ ИНТЕРФЕРОМЕТРА ФАБРИ — ПЕРО

ПОЛНЫЙ РАЗБОР ПРОГРАММЫ

Всё, что используется в программе, кроме основ Python: физика, математика, numpy, Qt, рисование, анимация, статистика, тесты, сборка программ, GitHub и документы

Пособие написано для того, кто знает основы Python: переменные, числа и строки, списки и словари, `if`, `for`, `while`, функции через `def`. Всё остальное объясняется с нуля: каждая библиотека, каждый приём, каждая формула и каждая строчка файлов сборки. Части можно читать по отдельности. Внутри каждой части сначала идёт идея простыми словами, потом подробности, потом настоящий код программы с разбором.

Фрагменты в рамке с зелёной полосой взяты прямо из файлов программы и всегда совпадают с ними. Фрагменты с серой полосой — учебные примеры.

Москва, 2026

СОДЕРЖАНИЕ

Часть 0. Карта программы: из чего она состоит и как её запускать
Часть 1. Python сверх основ: модули, классы, функции, строки, файлы
Часть 2. numpy: массивы и вычисления без циклов
Часть 3. Физика: от волны до колец, спектра и разрешения
Часть 4. Вычислительная математика программы: как она сама меряет графики
Часть 5. Погрешности, метод наименьших квадратов и режим лабораторной работы
Часть 6. Qt: окно, виджеты, сигналы, события, таймеры, настройки
Часть 7. Визуал: цвета, темы, стили, рисование графиков и колец
Часть 8. Анимация, путь данных и скорость
Часть 9. Сохранение: PNG, CSV, буфер обмена, настройки, журнал
Часть 10. Автоматические проверки (тесты)
Часть 11. Упаковка: как из кода получаются программы для Windows, macOS и Linux
Часть 12. git и GitHub Actions: хранение кода и автоматическая сборка
Часть 13. Как сделаны методички
Часть 14. Если что-то не работает
Часть 15. Словарь терминов

ЧАСТЬ 0. КАРТА ПРОГРАММЫ

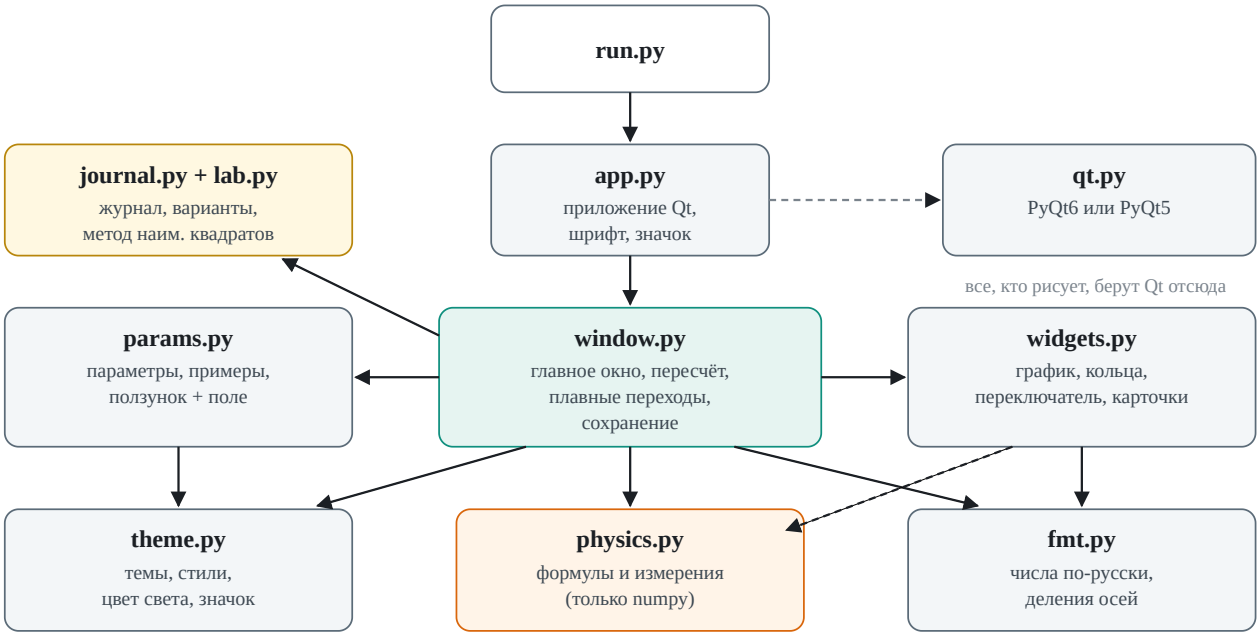
0.1. Что делает программа одной фразой

Программа берёт параметры интерферометра (длину волны λ, зазор d, отражение зеркал R и другие), считает по формулам физики, сколько света проходит через прибор, и рисует три картинки: график пропускания T(λ), картину колец и разрез колец по радиусу. Потом она сама «меряет» получившиеся картинки, как это сделал бы человек с линейкой, и сравнивает результат с формулами. В режиме лабораторной работы часть параметров скрыта, и студент находит их сам по своим измерениям.

0.2. Файлы проекта

Файл или папка	Что внутри	Нужен ли Qt
`run.py`	Запуск программы. Этот же файл берёт сборщик PyInstaller	да
`fabry_perot/__init__.py`	Делает папку пакетом Python; хранит номер версии и название	нет
`fabry_perot/__main__.py`	Позволяет запустить командой `python -m fabry_perot`	да
`fabry_perot/physics.py`	Все формулы физики и измерения по графикам	нет, только numpy
`fabry_perot/lab.py`	Варианты лабораторной, метод наименьших квадратов, серии измерений	нет
`fabry_perot/fmt.py`	Запись чисел по-русски, деления на осях	нет
`fabry_perot/qt.py`	Подключение PyQt6 или PyQt5, мелкие различия между ними	да
`fabry_perot/theme.py`	Цвета тёмной и светлой темы, таблица стилей, цвет света, значок	да
`fabry_perot/widgets.py`	Свои элементы окна: график, картина колец, переключатель, карточки	да
`fabry_perot/params.py`	Список параметров, готовые примеры, строка «ползунок + поле»	да
`fabry_perot/journal.py`	Журнал измерений справа	да
`fabry_perot/window.py`	Главное окно: собирает всё вместе, плавные переходы, сохранение	да
`fabry_perot/app.py`	Создание приложения Qt и функция `main()`	да
`tests/`	Автоматические проверки (часть 10)	частично
`requirements*.txt`	Какие библиотеки и каких версий ставить (часть 11.8)	—
`.github/workflows/build.yml`	Инструкция для GitHub: как проверить и собрать программы (часть 12)	—
`packaging/`	Файлы для сборки: сведения о версии для Windows, ярлык и установщик для Linux	—
`assets/`	Значок программы в трёх форматах: PNG, ICO (Windows), ICNS (macOS)	—
`tools/`	Вспомогательные программы: рисование значка, сборка методичек	—
`docs/`	Методички в PDF и их исходники	—

Кто кого использует, показано на схеме. Стрелка «A → B» значит «A импортирует B». Самые нижние модули ничего не знают об окне: `physics.py` можно взять и использовать в другой программе или в тестах.



0.3. Как запустить из исходников

```
pip install -r requirements.txt     # один раз: ставит numpy и PyQt6
python run.py                       # запуск
python -m fabry_perot               # то же самое, другим способом
python -m pytest tests              # запуск проверок (нужен pip install pytest)
```

`pip` — программа, которая скачивает библиотеки Python из интернета (с сайта PyPI) и ставит их. Ключ `-r файл` значит «поставь всё, что перечислено в этом файле».

0.4. Что происходит при запуске, по шагам

1. Python выполняет `run.py`. Там одна строчка: импортировать функцию `main` из `fabry_perot/app.py` и вызвать её.
2. При импорте `app.py` по цепочке загружаются все модули: `qt.py` выбирает PyQt6 или PyQt5, `theme.py` создаёт объект темы и так далее.
3. `main()` создаёт приложение Qt (`create_app`), затем главное окно `MainWindow()`. Конструктор окна строит все кнопки и графики и вызывает `restore()`: тот достаёт параметры прошлого запуска и делает первый расчёт `recalc()`.
4. `window.show()` показывает окно, `app.exec()` запускает «цикл событий»: программа ждёт действий пользователя и реагирует на них (часть 6.2). Пока окно открыто, выполнение «висит» внутри `app.exec()`.
5. Когда окно закрывают, вызывается `closeEvent`: он сохраняет параметры, после чего `app.exec()` возвращает код завершения, и `sys.exit` заканчивает программу.

ЧАСТЬ 1. PYTHON СВЕРХ ОСНОВ

Всё, что в программе встречается помимо переменных, циклов, условий и простых функций

1.1. Модули и пакеты

Модуль — это любой файл `.py`. **Пакет** — папка с модулями, в которой лежит файл `__init__.py` (он может быть и пустым). Наша папка `fabry_perot` — пакет.

Команда `import` выполняет файл модуля один раз (при первом импорте) и даёт доступ к тому, что в нём определено. Второй импорт того же модуля ничего не выполняет заново — Python помнит уже загруженные модули в словаре `sys.modules`. Поэтому объект `THEME` в `theme.py` существует в одном экземпляре, сколько бы модулей его ни импортировало.

```
import math                          # весь модуль; обращение: math.sqrt(2)
import numpy as np                   # весь модуль под коротким именем: np.sqrt(2)
from math import sqrt                # одно имя из модуля: sqrt(2)
from .qt import Qt, QtWidgets        # точка = «из этого же пакета» (относительный импорт)
from . import APP_TITLE              # имя из __init__.py этого пакета
from fabry_perot.app import main     # абсолютный импорт: пакет.модуль
```

Относительный импорт (с точкой) работает только внутри пакета. Внутри `fabry_perot/` все модули импортируют друг друга с точкой: так пакет можно переименовать или переложить, и ничего не сломается. А `run.py` лежит снаружи пакета, поэтому пишет полное имя `fabry_perot.app`.

`if __name__ == "__main__"`

У каждого модуля есть переменная `__name__`. Если файл запустили напрямую (`python run.py`), она равна строке `"__main__"`; если его импортировали — имени модуля. Условие в конце `run.py` значит «запускай окно, только если меня запустили, а не импортировали».

`__main__.py` и `python -m`

Команда `python -m fabry_perot` говорит: «найди пакет `fabry_perot` и выполни его файл `__main__.py`». Ключ `-m` (module) удобен тем, что пакет находится по обычным правилам импорта, и относительные импорты внутри него работают.

Строка документации

Строка в тройных кавычках в самом начале модуля, функции или класса — **docstring**. Python сохраняет её (её показывает `help(функция)`), на работу программы она не влияет. В программе у каждой функции есть такая строка: что делает функция и почему именно так.

1.2. Классы и объекты

> **ПРОСТЫМИ СЛОВАМИ**
>
> Класс — это чертёж. Объект — изделие по чертежу. У изделия есть свои данные (атрибуты) и то, что оно умеет делать (методы). Например, чертёж «строка параметра» `ParamRow` один, а изделий по нему девять: для λ, d, Δd, R, A, n, f, экрана и δλ. У каждого свой ползунок и своё поле.

```
class Counter:
    step = 1                          # атрибут класса: общий для всех объектов

    def __init__(self, start):        # конструктор: вызывается при создании объекта
        self.value = start            # атрибут объекта: у каждого свой

    def tick(self):                   # метод
        self.value += self.step

c = Counter(10)                       # создание: Python вызывает __init__(c, 10)
c.tick()                              # то же, что Counter.tick(c)
print(c.value)                        # 11
```

- **`self`** — сам объект. Python подставляет его первым аргументом автоматически: вызов `c.tick()` превращается в `Counter.tick(c)`. Через `self.имя` объект хранит свои данные между вызовами методов.
- **`__init__`** — метод, который Python вызывает сам при создании объекта. Здесь задаются начальные значения атрибутов. Конструктор `MainWindow.__init__` занимает 60 строк: он строит всё окно.
- **Атрибут класса** записан прямо в теле класса, без `self`. Пример: `ParamRow.STEPS = 1000` — у всех ползунков по 1000 положений. У `StepSpin` атрибут класса `stepping = False` служит значением по умолчанию; когда метод пишет `self.stepping = True`, у объекта появляется свой атрибут, который «заслоняет» общий.

Наследование

Запись `class Plot(QtWidgets.QWidget):` значит: «Plot — это QWidget (элемент окна Qt), плюс мои дополнения». Plot получает все методы QWidget (показаться, изменить размер, перерисоваться…) и добавляет свои. Если в Plot написать метод с тем же именем, что у QWidget, он **переопределяет** (заменяет) родительский. Именно так программа рисует свои графики: Qt, когда нужно нарисовать виджет, вызывает его метод `paintEvent`; у QWidget он ничего не рисует, а у Plot — рисует график.

fabry_perot/params.py — StepSpin

```
class StepSpin(QtWidgets.QDoubleSpinBox):
    stepping = False

    def stepBy(self, steps):
        self.stepping = True
        super().stepBy(steps)
        self.stepping = False
```

`super()` — «родительский класс». `super().stepBy(steps)` вызывает настоящий метод `QDoubleSpinBox.stepBy`, который и меняет число. Наш метод только оборачивает его: ставит флажок до и снимает после. Пока флажок стоит, поле шлёт сигнал «значение изменилось», и строка параметра видит, что это был шаг стрелкой (часть 8.3). В каждом конструкторе наследника первой строкой стоит `super().__init__()`: сначала должна отработать «родительская» подготовка объекта (у виджета Qt это создание настоящего окна-элемента внутри библиотеки), а потом уже наша.

Особые методы и `@property`

fabry_perot/theme.py — Theme

```
class Theme:
    def __init__(self):
        self.p = DARK

    def set(self, name):
        self.p = LIGHT if name == "light" else DARK

    @property
    def name(self):
        return self.p["name"]

    @property
    def dark(self):
        return self.p["name"] == "dark"

    def __getitem__(self, key):
        return self.p[key]

    def color(self, key, alpha=None):
        c = QColor(self.p[key])
        if alpha is not None:
            c.setAlpha(alpha)
        return c
```

- `__getitem__` — метод, который Python вызывает для записи с квадратными скобками. Благодаря ему пишется `THEME["bg"]`, как у словаря, хотя `THEME` — объект. Внутри он просто берёт значение из текущей палитры `self.p`.
- `@property` — **декоратор** (строка с `@` перед функцией, которая «оборачивает» функцию). Он превращает метод в атрибут «только для чтения»: пишется `THEME.dark` без скобок, а на самом деле каждый раз выполняется функция и проверяет имя палитры.
- Объект `THEME = Theme()` создаётся один раз при загрузке модуля. Все виджеты берут цвета из него в момент рисования. Сменить тему — значит сменить `THEME.p` и попросить окно перерисоваться.

Ещё особые методы, встречающиеся в программе, — это методы Qt с окончанием `Event` (`paintEvent`, `mousePressEvent`…). Они не особые для Python, но особые для Qt: Qt сам вызывает их, когда происходит событие (часть 6.6).

Простые классы-«записи»

Некоторые классы нужны только чтобы держать вместе несколько значений: `Curve` (формула кривой, цвет, имя), `Series` (название задания, подписи осей, подсказка…), `Variant` (скрытые величины варианта). Можно было бы хранить это в словарях, но с классом обращение короче (`s.title` вместо `s["title"]`) и можно добавить методы (`Curve.reading`).

1.3. Функции как значения

ПРОСТЫМИ СЛОВАМИ

В Python функция — такой же объект, как число или список. Её можно положить в переменную, передать в другую функцию, вернуть из функции. Главное отличие: `f` — это сама функция, а `f()` — результат её вызова.

В программе это используется постоянно:

- `ParamRow(grid, row, spec, self.param_changed)` — строке параметра передаётся метод окна. Строка запоминает его в `self.on_change` и вызывает `self.on_change(self.key, smooth)`, когда значение изменилось. Строка ничего не знает об окне, она знает только «кого позвать».
- `Journal(self.context, …)` — журнал получает функцию, которая возвращает текущие параметры окна.
- `button.clicked.connect(self.save_png)` — кнопке передаётся метод, который надо вызвать по щелчку (часть 6.5).
- `Curve(value, span, …)` — кривая на графике хранит не точки, а **две функции**: как посчитать значение в любой точке и как найти минимум и максимум на отрезке.

Связанный метод. `self.param_changed` (без скобок) — это метод, уже «привязанный» к конкретному окну. Когда его позже вызовут как `on_change("d", True)`, Python сам подставит нужный `self`.

`lambda`

`lambda аргументы: выражение` — короткая безымянная функция в одну строку. Запись `lambda x: x * 2` означает то же, что

```
def без_имени(x):
    return x * 2
```

Пример из программы: `key=lambda s: abs(math.log(s / raw))` в `fmt.ticks` — функция, по которой `min` выбирает «самый близкий» шаг делений.

Замыкания

Функция, определённая внутри другой функции, видит переменные внешней — даже после того, как внешняя закончилась. Это называется **замыкание**.

fabry_perot/window.py — recalc (фрагмент): кривые разреза колец

```
    share = len(lines)

    def sin_out(x_mm):
        r = x_mm * 1e-3
        return r / np.hypot(r, L)

    def ring_curve(wl, name):
        if rho > 0:
            def value(x_mm):
                r = np.asarray(x_mm, dtype=float) * 1e-3
                return screen_mean(np.maximum(r - rho, 0.0), r + rho, wl, d, n, L, F, tmax) /
share
            return Curve(value, sampled_span(value), line_color(wl * 1e9), name if two else "I /
I₀")

        def value(x_mm):
            return airy(phase(wl, d, n, sin_out(x_mm)), F, tmax) / share

        def span(edges):
            lo_, hi_ = airy_range(phase(wl, d, n, sin_out(edges)), F, tmax)
            return lo_ / share, hi_ / share
        return Curve(value, span, line_color(wl * 1e9), name if two else "I / I₀")

    curves = [ring_curve(wl, name) for wl, name in lines]
```

```
    if two:
        curves.append(self.sum_curve(list(curves)))
```

Здесь `value` и `span` пользуются переменными `d, n, L, F, tmax, rho, share, wl` из `recalc`. Функции «уносят» эти значения с собой в объект `Curve`, и график потом вызывает их, когда нужно нарисовать кривую при любом масштабе.

> **ЛОВУШКА**
>
> **lambda в цикле.** Замыкание запоминает не значение, а саму переменную. Если в цикле написать `button.clicked.connect(lambda: self.apply_preset(values))`, то все кнопки после цикла будут видеть **последнее** значение `values`. Поэтому в программе написано `lambda _, v=values, s=dlam: self.apply_preset(v, s)`: значения по умолчанию аргументов вычисляются в момент создания функции, так что у каждой кнопки свои `v` и `s`. Первый аргумент `_` — это `checked`, который шлёт сигнал `clicked`; он не нужен, а подчёркивание по договорённости означает «не используется». Тот же приём в журнале: `def value(x, a=a, b=b)`.

Ещё одна ловушка: список, содержащий сам себя

В `recalc` написано `curves.append(self.sum_curve(list(curves), sigma))`. Функция суммы складывает все кривые списка. Если передать сам список `curves`, то после `append` в нём окажется и кривая суммы, и она попробует сложить сама себя — бесконечно (ошибка `RecursionError`). `list(curves)` делает копию списка в момент вызова, и кривая суммы видит только две исходные кривые.

1.4. Распаковка, аргументы, словари

```
key, name, unit, lo, hi, value, decimals, step, tip, log = spec   # 10 элементов кортежа → 10 переменных
a, b = b, a                                                       # обмен значений
module, *names = "window.MainWindow.recalc".split(".")            # module="window", names=[…, …]
for button, values, dlam in self.preset_buttons: ...               # распаковка прямо в цикле
for i, (_, name) in enumerate(TILES): ...                          # вложенная распаковка
```

- **Звёздочка при распаковке** собирает «всё остальное» в список.
- `*args` в определении функции собирает лишние позиционные аргументы в кортеж: `def schedule(self, *args)` можно вызвать с любым числом аргументов — это нужно, потому что разные сигналы Qt передают разное число значений, а функции они не нужны.
- **Звёздочка при вызове** раскладывает список в отдельные аргументы: `LabDialog(self, *self.student)` то же, что `LabDialog(self, self.student[0], self.student[1])`.
- **Две звёздочки** раскладывают словарь в именованные аргументы: `p.chromium.launch(**{"executable_path": exe})` = `launch(executable_path=exe)`.
- **Именованные аргументы и значения по умолчанию**: `def num(x, digits=4)` можно вызвать как `num(x)` или `num(x, 3)`, или `num(x, digits=3)`.

- **Словарь с добавками**: `dict(BASE, lam=632.8, d=5.0, R=0.9)` — копия словаря `BASE`, в которую добавлены (или заменены) ключи. Так все примеры получают общие значения `n = 1, f = 200` и т. д.
- **`dict.get(key, default)`** — значение по ключу или default, если ключа нет (без ошибки). `dict.pop(key, None)` — удалить ключ, если он есть.

1.5. Короткие приёмы, которые встречаются в коде

Запись	Что значит
`x if условие else y`	условное выражение: одно из двух значений
`[f(x) for x in xs]`	список из значений `f(x)` для всех x (генератор списка)
`[x for x in xs if x > 0]`	то же с фильтром
`{s.key: s for s in SERIES}`	словарь по тому же принципу (так сделан `BY_KEY`)
`{i.row() for i in …}`	множество: без повторов (номера выбранных строк журнала)
`sum(x * y for x, y in zip(xs, ys))`	сумма без промежуточного списка; `zip` идёт по двум спискам сразу
`enumerate(список)`	пары (номер, элемент)
`any(…)`	True, если хоть один элемент истинен
`min(варианты, key=функция)`	элемент, у которого функция наименьшая
`sorted(…, reverse=True)`	отсортированная копия по убыванию
`a or b`	a, если оно «истинно» (не 0, не пусто, не None), иначе b. `width or fsr` — ширина, а если её нет, расстояние между пиками
`(list(m[:2]) + [None, None])[:2]`	ровно два элемента: первые два из m, недостающие — None
`x is None`	проверка «это именно None» (а не 0 или пустая строка)
`v is values`	тот же самый объект, а не просто равный (так ищется нажатая кнопка примера)
`1 << 30`	сдвиг битов влево: 2^{30}
`float(x).is_integer()`	число целое (2.0 → True)
`1e-9, 0.05e-12`	научная запись: $1\cdot10^{-9}$, $0{,}05\cdot10^{-12}$
`range(1, 31)`	числа 1…30 (правая граница не входит)

1.6. Строки и f-строки

f-строка — строка с буквой `f` перед кавычкой, в которую в фигурных скобках вставляются значения: `f"v{__version__}"` → `v2.0.0`. После двоеточия внутри скобок пишется **формат**:

Формат	Пример	Результат
`:.3f`	`f"{3.14159:.3f}"`	3.142 — ровно 3 знака после точки
`:,.2f`	`f"{15803.5:,.2f}"`	15,803.50 — запятая разделяет тысячи
`:g`	`f"{0.0100:g}"`	0.01 — без лишних нулей
`:.6g`	`f"{632.80001:.6g}"`	632.8 — не больше 6 значащих цифр
`:.2f внутри`	`f"{t:.2f}"`	используется в градиенте радуги
вложенный формат	`f"{x:.{k}f}"`	число знаков берётся из переменной k

Англоязычный формат пишет точку и запятую «наоборот» по сравнению с русским. Поэтому `fmt.num` сначала заменяет запятую-разделитель тысяч на неразрывный пробел, а потом точку на запятую:

fabry_perot/fmt.py — num

```
def num(x, digits=4):
    if x is None:
        return "—"
    if isinstance(x, (int, np.integer)):
        return f"{x:,}".replace(",", " ")
    if not math.isfinite(x):
        return "—"
    if x == 0:
        return "0"
    power = math.floor(math.log10(abs(x)))
    if -3 <= power < 6:
        text = f"{x:,.{max(digits - 1 - power, 0)}f}".replace(",", " ")
    else:
        text = f"{x / 10 ** power:.{digits - 1}f}·10" + str(power).translate(SUP)
    return text.replace(".", ",").replace("-", "−")
```

- `math.log10(abs(x))` — десятичный логарифм: сколько цифр до запятой минус один. `floor` — округление вниз. Для 4708000 степень равна 6.
- Если степень от −3 до 5, число пишется обычно, а знаков после запятой ровно столько, чтобы всего было 4 значащие цифры: `digits − 1 − power`. Для 40,04: power = 1, знаков 4 − 1 − 1 = 2.
- Иначе — через степень десяти. `str.maketrans` делает таблицу замены символов, а `translate` применяет её: цифры степени превращаются в верхние индексы $^{0}{}^{1}{}^{2}$…
- Минус `-` заменяется на настоящий типографский минус `−` (он длиннее дефиса).
- `isinstance(x, (int, np.integer))` — проверка типа: целое число Python или целое numpy.

Другие методы строк: `replace(a, b)` — заменить все a на b; `strip()` — убрать пробелы по краям; `lstrip("-")` — убрать минусы слева; `split(",")` — разрезать по запятой в список; `" ".join(parts)` — склеить список, вставив между элементами разделитель; `upper()` — заглавными буквами; `endswith(".png")` — кончается ли строка на…

Unicode. Строки Python хранят любые символы: λ, δ, ², ₁, «», ≈, ⌀. Их можно писать прямо в коде, потому что файлы сохранены в кодировке UTF-8.

1.7. Исключения: `try / except`

> **ПРОСТЫМИ СЛОВАМИ**
> Исключение — это «авария» в программе: деление на ноль, нет файла, не тот тип. Если его не поймать, программа останавливается с сообщением об ошибке. Блок `try … except` говорит: «попробуй; если случится вот такая авария — сделай вот это вместо остановки».

Где	Какое исключение	Зачем ловится
`qt.py`	`ImportError`	PyQt6 не установлен (Windows 7) — берём PyQt5
`app.py`	`AttributeError`	в старом Qt 5 нет функции PassThrough — просто пропускаем
`window.restore`	`TypeError`, `ValueError`	в файле настроек мусор вместо числа — оставляем значение по умолчанию
`window.write_csv`	`OSError`	файл не записался (нет прав, открыт в Excel) — показываем сообщение
`journal.load_json`	`TypeError`, `ValueError`	сохранённый журнал испорчен — начинаем с пустого

`except OSError as error:` — поймать и положить само исключение в переменную, чтобы показать его текст. `pass` — пустая команда «ничего не делать». `raise KeyError(…)` — самому вызвать исключение (так делает сборщик методичек, если не нашёл функцию).

1.8. Файлы: `with open(…)`

```
with open(path, "w", newline="", encoding="utf-8-sig") as file:
    ...                                 # пишем в file
# здесь файл уже закрыт, даже если внутри случилась ошибка
```

- `"w"` — открыть на запись (старое содержимое стирается); `"r"` — на чтение.
- `encoding` — как символы превращаются в байты. `utf-8` — стандарт; `utf-8-sig` добавляет в начало файла три невидимых байта-метки (BOM). По этой метке Excel понимает, что файл в UTF-8, и показывает русские буквы, а не «кракозябры».
- `newline=""` — требование модуля `csv`: он сам ставит переносы строк, а без этого на Windows между строками появлялись бы пустые.
- `with` — «менеджер контекста»: гарантирует закрытие файла при выходе из блока.

1.9. Модули стандартной библиотеки

Они идут вместе с Python, ставить их не нужно.

Модуль	Что из него берётся	Где
`math`	`floor, ceil` (округление вниз/вверх), `log10, log`, `sqrt`, `hypot(a, b)` = √(a²+b²), `asin` (арксинус), `radians` (градусы → радианы), `isfinite` (не бесконечность и не nan), `pi`	везде
`random`	`random.Random(зерно)` — свой генератор случайных чисел; `uniform(a, b)`, `choice(список)`, `shuffle(список)`	`lab.Variant`
`json`	`dumps` (объект → текст), `loads` (текст → объект)	журнал
`csv`	`csv.writer(file, delimiter=";")`, `writerow(список)`	сохранение таблицы
`os`	`os.environ` (переменные окружения), `os.path.join` (склейка пути), `os.makedirs`	`qt.py`, tools
`sys`	`sys.argv` (аргументы командной строки), `sys.exit(код)`, `sys.executable`, `sys.path`	`app.py`, tools
`re`	регулярные выражения — поиск по шаблону в тексте	сборка методичек
`ast`	разбор кода Python на части (найти функцию по имени)	сборка методичек
`glob, subprocess, html, textwrap`	поиск файлов по маске, запуск другой программы, экранирование < >, снятие отступа	сборка методичек

Переменная окружения — именованная строка, которую операционная система передаёт каждой запущенной программе. Например, `FABRY_QT=5 python run.py` запустит программу с переменной `FABRY_QT`, равной `"5"`, и `qt.py` возьмёт PyQt5. `QT_QPA_PLATFORM=offscreen` говорит Qt рисовать окна в памяти, без экрана (так работают тесты на серверах).

`sys.argv` — список слов командной строки: для `FabryPerot.exe --selftest` это `["FabryPerot.exe", "--selftest"]`. `main()` проверяет, есть ли там `--selftest`.

ЧАСТЬ 2. NUMPY

Библиотека для быстрых вычислений сразу над тысячами и миллионами чисел

2.1. Зачем numpy

> **ПРОСТЫМИ СЛОВАМИ**
>
> Обычный цикл Python обрабатывает около 10–30 миллионов простых операций в секунду: каждая операция проверяет типы, создаёт объекты и т. п. numpy хранит числа сплошным блоком памяти одного типа и обрабатывает весь блок кодом на C за одну команду. Получается в 50–200 раз быстрее. Картинка колец 700 × 700 — это почти полмиллиона пикселей; обойти их циклом Python — около секунды, numpy — несколько миллисекунд.

```
import numpy as np
x = np.linspace(0, 1, 5)       # массив [0.  , 0.25, 0.5 , 0.75, 1.  ]
y = np.sin(x) * 2 + 1          # одна строка — вычисление для всех 5 чисел сразу
# без numpy пришлось бы писать:
y = [math.sin(v) * 2 + 1 for v in xs]
```

Такой стиль называется **векторизация**: вместо цикла «для каждого числа» пишется формула для целого массива. Функции `physics.py` написаны так, что работают и с одним числом, и с массивом: в `phase(lam, d, n)` можно передать массив длин волн и получить массив фаз.

2.2. Массив: форма и тип

- **Форма** (`shape`) — размеры массива. Одномерный из 5 чисел: `(5,)`. Картинка 400 × 300: `(300, 400)` — сначала строки (высота), потом столбцы (ширина).
- **Тип** (`dtype`) — какие числа внутри. Все элементы одного типа:
  - `float64` — обычные дробные числа (≈ 16 значащих цифр), тип по умолчанию;
  - `float32` — дробные с ≈ 7 цифрами, вдвое меньше памяти и быстрее; используется для растеризации графика, где точность пикселя не нужна;
  - `uint8` — целые 0…255 (один байт) — степень прозрачности пикселя;
  - `uint32` — целые 0…4 294 967 295 (четыре байта) — цвет пикселя целиком;
  - `intp` — целые для номеров элементов (индексов).
- `a.astype(np.uint8)` — новый массив другого типа (дробная часть отбрасывается, поэтому перед этим прибавляют 0,5 — так получается округление).

2.3. Как создаются массивы

Функция	Результат	Где в программе
`np.array([1, 2, 3])`	массив из списка	цвет линии
`np.zeros(n)`, `np.zeros((n, 3))`	n нулей; таблица n × 3 из нулей	цвета колец
`np.arange(a, b, шаг)`	a, a+шаг, … (до b, не включая)	номера колец, пикселей
`np.linspace(a, b, n)`	n равноотстоящих чисел от a до b включительно	точки графика

Чем отличаются `arange` и `linspace`: у первого задан шаг, у второго — число точек. `linspace(x0, x1, width + 1)` даёт ровно `width + 1` границ для `width` столбцов пикселей.

2.4. Поэлементные операции

`+ − * / **` между массивами одной формы действуют попарно: `a * b` — массив произведений. Между массивом и числом — число применяется к каждому элементу. Функции numpy тоже поэлементные:

Функция	Что делает
`np.sin, np.tan, np.arctan, np.sqrt, np.exp, np.log`	как в math, но для массивов
`np.floor, np.abs, np.hypot`	округление вниз, модуль, $\sqrt{a^2+b^2}$
`np.minimum(a, b), np.maximum(a, b)`	попарный минимум и максимум двух массивов
`np.clip(a, lo, hi)`	обрезать значения: меньше lo → lo, больше hi → hi
`np.where(условие, a, b)`	поэлементный if: где условие верно — из a, иначе из b
`np.divide(a, b, out=…, where=b != 0)`	деление только там, где делитель не ноль; в остальных местах — значение из out

`np.where` — главный заменитель `if` в векторном коде. В `airy_range`: `np.where(has_peak, tmax, np.maximum(ta, tb))` — «если на отрезке есть пик, максимум равен Tmax, иначе — большему из концов», сразу для тысяч отрезков.

Сравнения тоже поэлементные и дают **массив True/False** (булев массив): `T < half`. Такие массивы объединяются знаками `&` (и), `|` (или) — не словами `and`/`or`, которые для массивов не работают. Каждое сравнение при этом берут в скобки: `(s2 > 0) & (s2 < 1)`.

2.5. Индексы, срезы, маски

```
y = np.array([0, 3, 1, 5, 2])
y[1:-1]           # [3, 1, 5]   — без первого и последнего («середина»)
y[:-2]            # [0, 3, 1]   — «левые соседи» середины
y[2:]             # [1, 5, 2]   — «правые соседи» середины
y[1:-1] > y[:-2]  # [True, False, True] — середина больше левого соседа?
y[[3, 1]]         # [5, 3]      — выбор по списку номеров («причудливые индексы»)
y[y > 1]          # [3, 5, 2]   — выбор по маске
np.flatnonzero(y > 1)   # [1, 3, 4] — номера, где условие верно
```

Так `find_peaks` ищет максимумы без цикла: точка — максимум, если она больше левого соседа и не меньше правого. Сравнение трёх сдвинутых срезов проверяет это условие сразу для всех точек, `flatnonzero` даёт их номера, а `+ 1` поправляет сдвиг (срез `y[1:-1]` начинается с элемента 1).

fabry_perot/physics.py — find_peaks

```python
def find_peaks(x, y):
    i = np.flatnonzero((y[1:-1] > y[:-2]) & (y[1:-1] >= y[2:])) + 1
    left, mid, right = y[i - 1], y[i], y[i + 1]
    bend = left - 2 * mid + right
    shift = np.divide(0.5 * (left - right), bend, out=np.zeros(len(i)), where=bend != 0)
    return i, x[i] + shift * (x[1] - x[0]), mid - 0.25 * (left - right) * shift
```

Выбор по массиву номеров `table[index]` работает и для двумерного `index`: результат имеет форму `index`, а значения берутся из `table`. Так картинка колец получает цвет каждого пикселя по его расстоянию до центра (часть 7.9). Запись `grid[np.arange(len(center)), best]` берёт из каждой строки таблицы `grid` элемент со своим номером `best`.

2.6. Broadcasting — «растягивание» форм

> **ПРОСТЫМИ СЛОВАМИ**
>
> Если сложить столбец из 3 чисел и строку из 4 чисел, numpy получит таблицу 3 × 4: каждый элемент столбца сложится с каждым элементом строки. Это и есть broadcasting. Так строятся двумерные картинки без двойного цикла.

```python
rows = np.arange(3)[:, None]      # форма (3, 1): столбец [[0], [1], [2]]
cols = np.arange(4)[None, :]      # форма (1, 4): строка [[0, 1, 2, 3]]
rows * 10 + cols                  # форма (3, 4):
# [[ 0,  1,  2,  3],
#  [10, 11, 12, 13],
#  [20, 21, 22, 23]]
```

`[:, None]` добавляет ось длины 1 справа (из строки делает столбец), `[None, :]` — слева. Правило: оси длины 1 «растягиваются» до длины соответствующей оси другого массива.

Примеры в программе:

- `RingView.make_image`: `np.hypot(c[None, :] + cx, c[:, None] + cy)` — расстояние от центра колец для каждого пикселя квадратной картинки: `c[None, :]` — координата x столбца, `c[:, None]` — координата y строки.
- `Plot.draw_curve`: `rows` — столбец номеров строк пикселей, `top` и `bottom` — строки «верх и низ кривой в каждом столбце»; `np.minimum(rows + 1, bottom + half)` даёт сразу всю картинку.
- `measure_rings`: `center[:, None] + step * offsets[None, :]` — вокруг каждого из сотни найденных колец сетка из 33 точек, всего таблица 100 × 33.

2.7. Сводные функции и форма

- `a.max(), a.min(), a.sum()` — по всему массиву.
- `a.max(axis=1)` — по каждой строке отдельно (ось 1 — вдоль строки). Результат — столбец максимумов.

- `np.argmax(a, axis=1)` — **номер** наибольшего элемента в каждой строке.
- `a.ravel()` — вытянуть таблицу в одну строку; `a.reshape(форма)` — обратно в таблицу.
- `a.copy()` — копия. Срез `a[:10]` в numpy — не копия, а «окно» в тот же массив: изменения в нём меняют исходный массив. Поэтому в `measure_rings` написано `peaks[:refine].copy()`.

2.8. Прочие функции numpy из программы

Функция	Что делает	Где
`np.searchsorted(pos, x)`	в отсортированном массиве находит место, куда встал бы x	пики слева и справа от λ_1
`np.interp(x, xp, fp)`	линейная интерполяция: значение между известными точками	ширина пика на половине высоты
`np.convolve(a, k, mode="valid")`	свёртка: каждое значение заменяется взвешенной суммой соседей с весами k	свечение колец
`np.pad(a, k, mode="reflect")`	добавить k значений с каждого края, зеркально отражая	свечение: чтобы у краёв было что сворачивать
`a.tobytes()`	все числа массива подряд как байты	передача картинки в Qt

Вычисления «на месте»

`line -= x`, `clear *= 1 - fill`, `np.clip(line, 0, 1, out=line)` меняют существующий массив, не создавая нового. Для картинки в полмиллиона чисел это экономит время на выделение памяти. Запись `a = a - x` создала бы новый массив, а старый выбросила.

Битовые операции для цвета

Цвет пикселя в Qt — одно 32-битное число: 8 бит прозрачности, 8 красного, 8 зелёного, 8 синего: `0xAARRGGBB`. Собрать его из четырёх чисел 0…255 можно сдвигами: `(a << 24) | (r << 16) | (g << 8) | b`. Сдвиг `<< 16` умножает на 2^{16} — число «переезжает» на 16 бит влево, а `|` (побитовое «или») складывает части, не мешая друг другу. Так сделаны `color_table` и таблица цветов колец.

2.9. Дробные числа в компьютере

- Число `float64` хранит около 16 значащих цифр. `0.1 + 0.2` даёт `0.30000000000000004`: десятичные дроби не всегда записываются точно в двоичном виде. Поэтому числа сравнивают не на равенство, а с допуском: `abs(a − b) < 1e-12`, или в тестах `pytest.approx`.
- **Потеря точности при вычитании близких чисел.** Порядок интерференции в центре $m_0 = 2nd/\lambda \approx 15\,800$. Если считать $\sin^2\theta = n^2 - (m\lambda/2d)^2$, вычитаются два числа около 1, отличающиеся в 5–6 знаке, и половина значащих цифр теряется. `ring_radii` переписывает формулу как `a·(m₀ − m)·(n + m·a)` (это разность квадратов $x^2 - y^2 = (x - y)(x + y)$): разность $m_0 - m$ считается точно, и радиусы получаются верными до последних знаков.

- `nan` («не число») получается, например, из 0/0 или √(−1) в numpy; `inf` — бесконечность. `math.isfinite(x)` проверяет, что x — обычное число.
- Порог `1e-9` в `fmt.ticks` (`math.ceil(lo / step − 1e-9)`) защищает от того, что 0,14 / 0,02 в компьютере равно 7,000000000000001, и округление вверх дало бы 8 вместо 7 — первое деление оси пропало бы. Небольшой запас убирает такие ошибки.

ЧАСТЬ 3. ФИЗИКА

От того, что такое волна, до каждой формулы в physics.py

3.1. Свет — волна

> **ПРОСТЫМИ СЛОВАМИ**
>
> Свет — это колебания электрического и магнитного поля, которые бегут в пространстве со скоростью $c \approx 3 \cdot 10^8$ м/с. Если «сфотографировать» волну, поле вдоль луча меняется как синусоида. Расстояние между соседними гребнями — **длина волны** λ. От длины волны зависит цвет.

- **Длина волны λ** видимого света — от 380 нм (фиолетовый) до 780 нм (тёмно-красный). 1 нм (нанометр) = 10^{-9} м, 1 пм (пикометр) = 10^{-12} м = 0,001 нм, 1 фм (фемтометр) = 10^{-15} м. Гелий-неоновый лазер светит на 632,8 нм (красный), зелёная лазерная указка — на 532 нм, натриевая лампа — на двух линиях 589,0 и 589,6 нм (жёлтый).
- **Частота** ν — сколько колебаний в секунду в данной точке: $\nu = c/\lambda$. Для 632,8 нм $\nu \approx 4{,}74 \cdot 10^{14}$ Гц.
- **Фаза** — «в какой части колебания» волна находится, в радианах. Один полный период — 2π радиан (360°). Когда волна проходит путь L, её фаза меняется на $2\pi L/\lambda$: каждая длина волны пути — ещё один полный оборот.
- Глаз и приёмники чувствуют не само поле, а **интенсивность** — среднюю мощность, пропорциональную квадрату амплитуды поля. Колебания 10^{14} раз в секунду не может отследить ни один прибор.

3.2. Интерференция

Когда в одну точку приходят две волны одинаковой частоты, их поля складываются. Результат зависит от разности фаз $\Delta\varphi$:

- $\Delta\varphi = 0, 2\pi, 4\pi\ldots$ — гребень на гребень, волны усиливают друг друга (**максимум**);
- $\Delta\varphi = \pi, 3\pi\ldots$ — гребень на впадину, волны гасят друг друга (**минимум**).

Для двух одинаковых волн интенсивность $I = 2I_0(1 + \cos\Delta\varphi)$: от 0 до $4I_0$. Это и есть интерференция. Разность фаз обычно возникает из-за **разности хода** — одна волна прошла путь длиннее другой на Δ, тогда $\Delta\varphi = 2\pi\Delta/\lambda$.

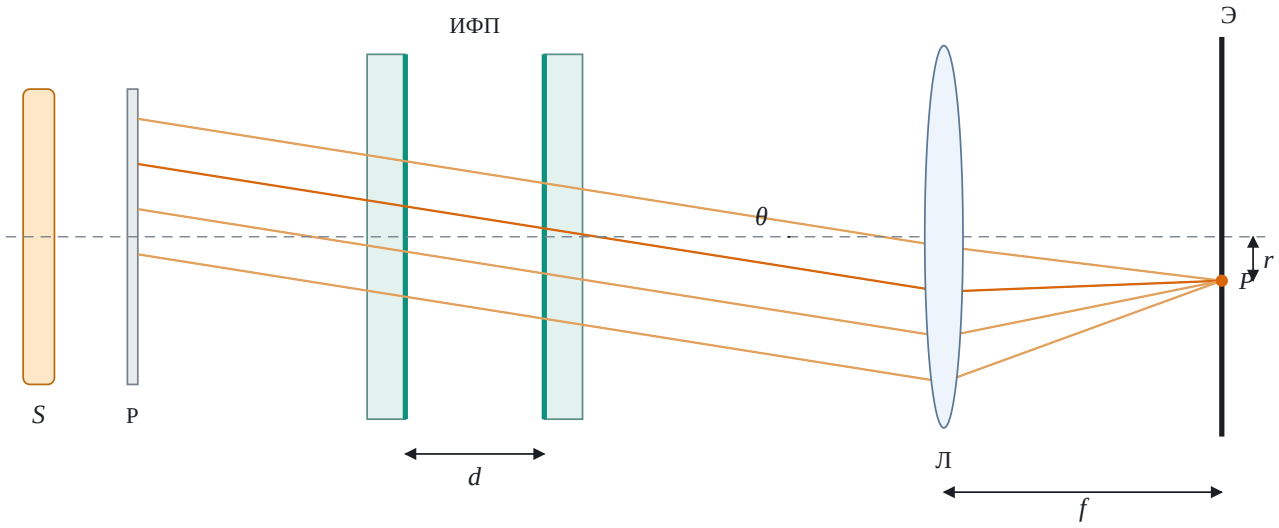
Как складывать волны: комплексные числа

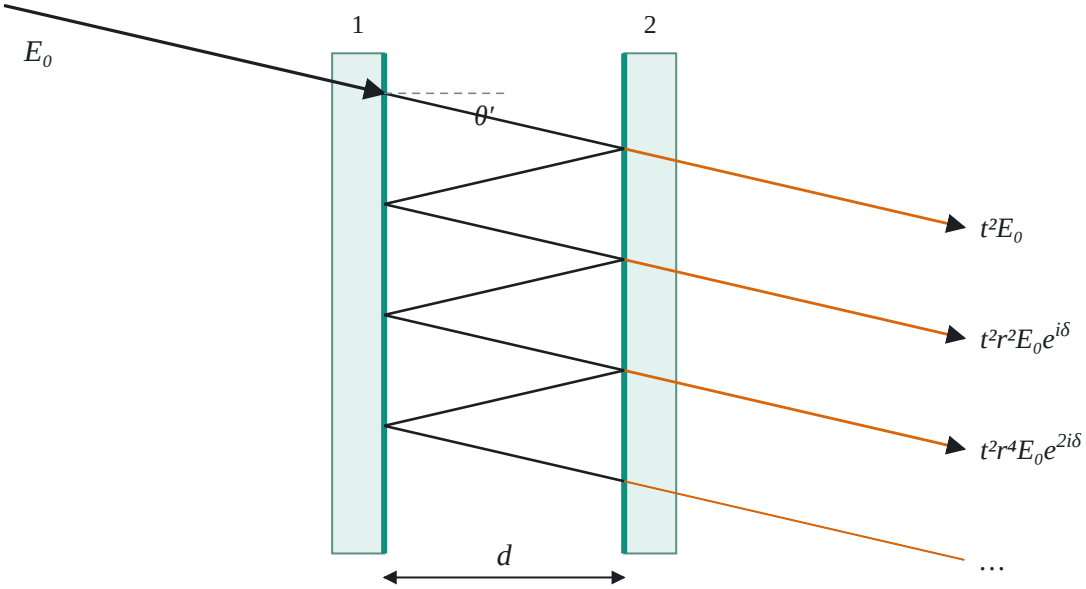
Складывать синусоиды с разными фазами неудобно. Физики записывают волну как комплексное число $E = a\,e^{i\varphi}$ — стрелку длины a, повёрнутую на угол φ. Складывать волны — значит складывать стрелки, а интенсивность — квадрат длины получившейся стрелки: $I = |E|^2$. Этого достаточно, чтобы понять вывод ниже. Тест `test_airy_matches_sum_of_beams` прямо так и делает: складывает 800 стрелок с помощью комплексных чисел numpy (`1j` — мнимая единица в Python) и сравнивает с формулой.

3.3. Устройство интерферометра

Два плоских частично прозрачных зеркала стоят параллельно на расстоянии d. Каждое зеркало отражает долю R интенсивности, поглощает долю A и пропускает долю $1 - R - A$. Свет

входит через первое зеркало и много раз ходит между зеркалами. На каждом отражении от второго зеркала часть света выходит наружу. Выходящие лучи параллельны, линза собирает их в одну точку своего фокального экрана, и там они интерферируют.





Каждый следующий выходящий луч прошёл зазор ещё два раза (туда и обратно), поэтому отстаёт от предыдущего на разность хода Δ = 2nd·cos θ′, где θ′ — угол луча внутри зазора, n — показатель преломления среды в зазоре (воздух ≈ 1, стекло ≈ 1,5). Показатель преломления показывает, во сколько раз свет в среде медленнее, чем в вакууме; путь L в среде эквивалентен пути nL в вакууме (**оптический путь**).

3.4. Формула Эйри: вывод

Пусть амплитуда входящей волны 1. Амплитудный коэффициент пропускания зеркала t (по интенсивности $t^2 = 1 - R - A$), амплитудный коэффициент отражения r ($r^2 = R$). Первый выходящий луч прошёл два зеркала: t^2. Второй дополнительно дважды отразился и отстал по фазе на δ: $t^2r^2e^{i\delta}$. Третий: $t^2r^4e^{2i\delta}$ и так далее. Сумма — геометрическая прогрессия со знаменателем $q = Re^{i\delta}$:

$$E = t^2\left(1 + q + q^2 + \ldots\right) = \frac{t^2}{1 - Re^{i\delta}}$$

(сумма бесконечной геометрической прогрессии $1 / (1 - q)$ при $|q| < 1$). Интенсивность — квадрат модуля:

$$T = |E|^2 = \frac{(1 - R - A)^2}{|1 - Re^{i\delta}|^2} = \frac{(1 - R - A)^2}{1 - 2R\cos\delta + R^2}$$

Знаменатель переписывается через $\cos\delta = 1 - 2\sin^2(\delta / 2)$: $1 - 2R + R^2 + 4R\sin^2(\delta / 2) = (1 - R)^2 + 4R\sin^2(\delta / 2)$. Поделив числитель и знаменатель на $(1 - R)^2$, получаем **формулу Эйри**:

$$T = \frac{T_{\max}}{1 + F\sin^2(\delta / 2)}, \qquad F = \frac{4R}{(1 - R)^2}, \qquad T_{\max} = \frac{(1 - R - A)^2}{(1 - R)^2} \tag{1}$$

fabry_perot/physics.py — airy

```
def airy(delta, F, tmax=1.0):
    return tmax / (1.0 + F * np.sin(delta / 2.0) ** 2)
```

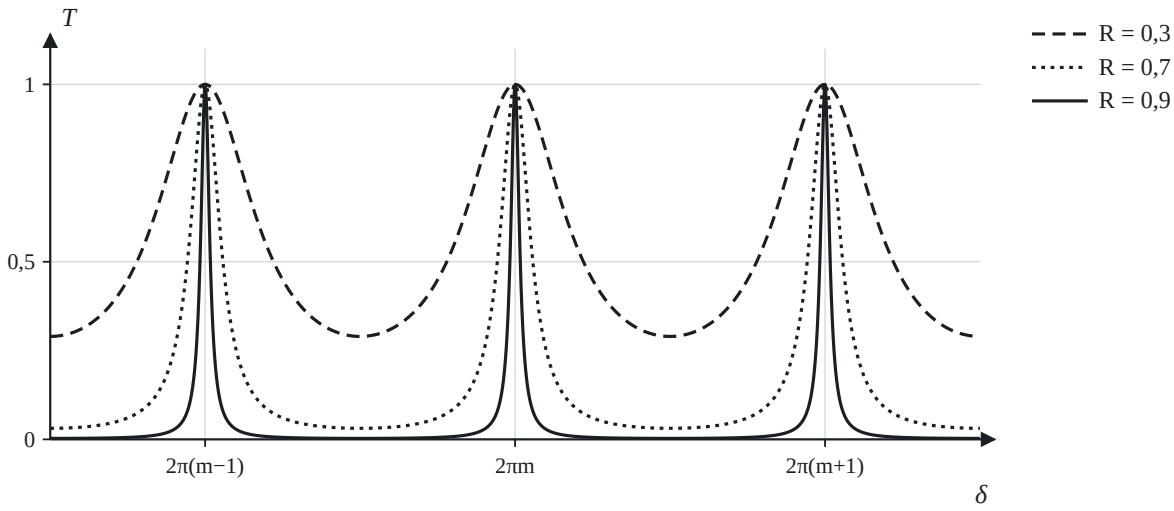
fabry_perot/physics.py — coefficient_f

```
def coefficient_f(R):
    return 4.0 * R / (1.0 - R) ** 2
```

fabry_perot/physics.py — peak_transmission

```
def peak_transmission(R, A=0.0):
    return (max(1.0 - R - A, 0.0) / (1.0 - R)) ** 2
```

- Когда $\delta = 2\pi m$ (m — целое), $\sin(\delta / 2) = 0$ и $T = T_{\max}$ — **пик** пропускания: все лучи приходят в фазе.
- Посередине между пиками $\sin^2 = 1$ и $T_{\min} = T_{\max} / (1 + F)$. Отношение $T_{\max} / T_{\min} = 1 + F$ называется **контрастом**. При R = 0,9 F = 360: провалы в 361 раз темнее пиков.
- Без поглощения (A = 0) $T_{\max} = 1$: на пике проходит **весь** свет, хотя каждое зеркало отражает 90 %. Это не ошибка: отражённые назад волны гасят друг друга, и энергии некуда деваться, кроме как пройти.
- С поглощением пики ниже: при R = 0,95 и A = 0,02 $T_{\max} = (0,03 / 0,05)^2 = 0,36$. Свет много раз ходит между зеркалами и каждый раз теряет долю A. `max(1.0 - R - A, 0.0)` защищает от R + A > 1 (зеркало не может отразить и поглотить больше 100 %).



3.5. Фаза δ и закон преломления

Разность фаз соседних лучей $\delta = 2\pi \cdot 2nd\cos\theta' \,/\, \lambda$. Удобнее выразить её через угол θ, под которым луч выходит из прибора в воздух: по закону Снеллиуса (закону преломления) $\sin\theta = n\sin\theta'$, отсюда $n\cos\theta' = \sqrt{n^2 - n^2\sin^2\theta'} = \sqrt{n^2 - \sin^2\theta}$:

$$\delta = \frac{4\pi d\sqrt{n^2 - \sin^2\theta}}{\lambda} \tag{2}$$

fabry_perot/physics.py — phase

```python
def phase(lam, d, n, sin_out=0.0):
    return 4.0 * np.pi * d * np.sqrt(n * n - sin_out * sin_out) / lam
```

Аргумент `sin_out` — это sin θ. По умолчанию 0: свет идёт перпендикулярно зеркалам, и тогда δ = 4πnd/λ. Для графика T(λ) в программе берётся именно этот случай (надпись «свет падает перпендикулярно»).

3.6. Порядок интерференции

Условие пика $\delta = 2\pi m$ с формулой (2) даёт $2d\sqrt{n^2 - \sin^2\theta} = m\lambda$: на удвоенной оптической толщине укладывается целое число m длин волн. Число m — **порядок интерференции**. В центре (θ = 0) дробный «порядок» $m_0 = 2nd \,/\, \lambda$; при λ = 632,8 нм, d = 5 мм $m_0 \approx 15\,803$. Функция `order_at` считает порядок в любой точке экрана — его показывает курсор над кольцами («m = …»).

3.7. Кольца

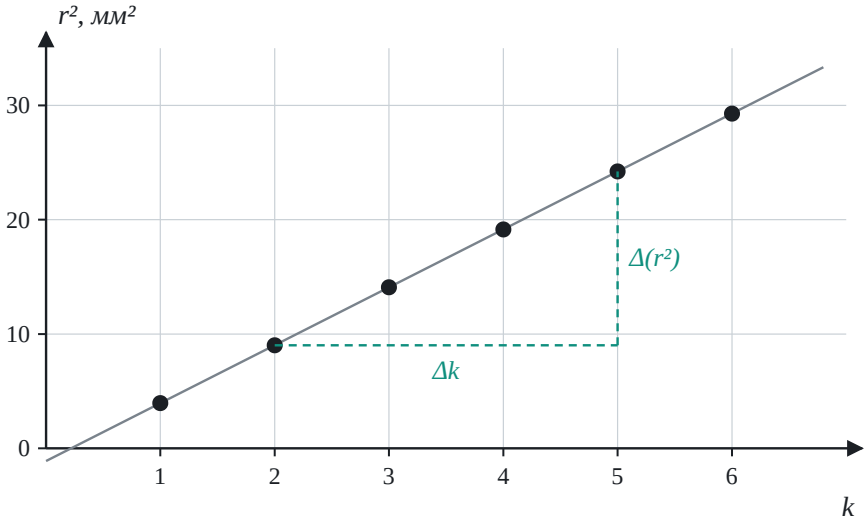
Если свет от протяжённого источника идёт под разными углами, у каждого угла свой δ. Условие пика выполняется на определённых углах θ_k, а из-за симметрии вокруг оси прибора одинаковый угол — это окружность. Линза с фокусным расстоянием f собирает лучи, идущие под углом θ, в точку на расстоянии $r = f\,\mathrm{tg}\,\theta$ от центра фокального экрана. Получаются концентрические светлые **кольца**.

- Порядок m убывает от центра к краю (cos θ уменьшается). Первое кольцо — наибольшее целое m, меньшее m_0.

- Для малых углов $\cos\theta \approx 1 - \theta^2 / 2$, $r \approx f\theta$. Подстановка в условие пика даёт

$$r_k^2 = \frac{f^2 n\lambda}{d}\,(k - 1 + \varepsilon) \tag{3}$$

где k — номер кольца от центра (1, 2, 3…), ε — дробная часть m_0 ($0 \le \varepsilon < 1$): насколько центр «не дошёл» до следующего кольца. Главное: **квадраты радиусов растут линейно с номером**, разность квадратов соседних колец одна и та же: $r_{k+1}^2 - r_k^2 = f^2 n\lambda / d$. Поэтому кольца сгущаются к краю, и поэтому в лабораторной строится график r^2 от k: по наклону находится λ.



fabry_perot/physics.py — ring_radii

```python
def ring_radii(lam, d, n, f, r_max, limit=500):
    m0 = 2 * n * d / lam
    a = lam / (2 * d)
    first = math.ceil(m0) - 1
    m = np.arange(first, max(first - limit, 0), -1, dtype=float)
    s2 = a * (m0 - m) * (n + m * a)
    s2 = s2[(s2 > 0) & (s2 < 1)]
    r = f * np.sqrt(s2 / (1 - s2))
    return r[r <= r_max]
```

Функция считает радиусы по **точной** формуле, без приближения малых углов: из $2d\sqrt{n^2 - s^2} = m\lambda$ получаем $s^2 = \sin^2\theta = n^2 - (m\lambda / 2d)^2$, потом $\operatorname{tg}\theta = s / \sqrt{1 - s^2}$ и $r = f\operatorname{tg}\theta$. Запись через $a(m_0 - m)(n + ma)$ — это та же разность квадратов, но без потери точности (часть 2.9). Фильтр `(s2 > 0) & (s2 < 1)` отбрасывает несуществующие углы.

3.8. Спектр пропускания T(λ) и область свободной дисперсии

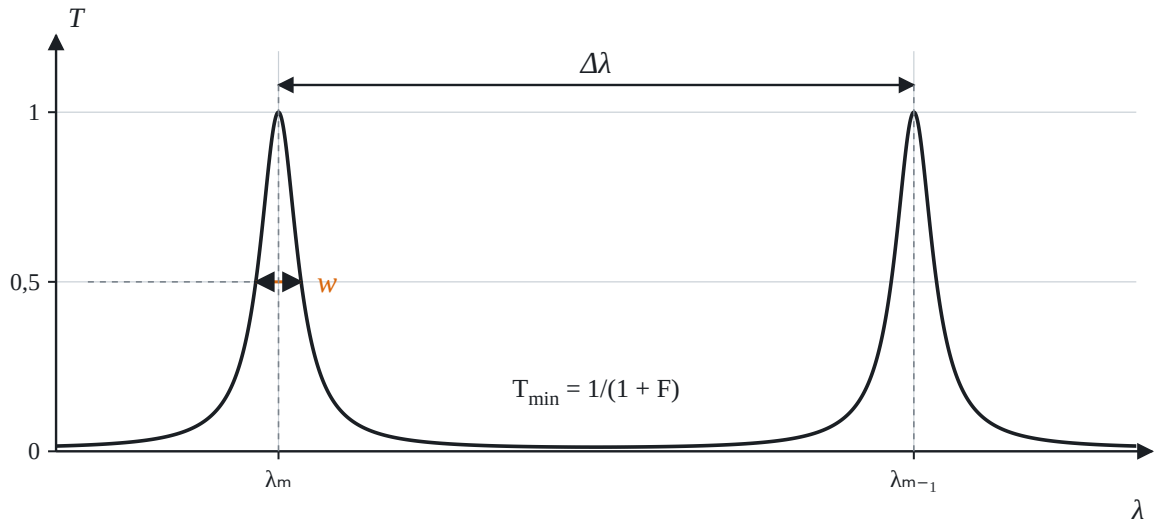
Если зафиксировать угол (θ = 0) и менять длину волны, δ = 4πnd/λ тоже меняется, и пропускание проходит через пики. Соседние пики — это порядки m и m+1. Расстояние между ними по длине волны:

$$\Delta\lambda = \frac{\lambda^2}{2nd}, \qquad \Delta\nu = \frac{c}{2nd} \tag{4}$$

Вывод: $m\lambda_m = 2nd$, $(m+1)\lambda_{m+1} = 2nd$; разность $\lambda_m - \lambda_{m+1} = \lambda / (m+1) \approx \lambda^2 / (2nd)$. По частоте всё точно: $\nu_m = mc / (2nd)$, шаг $c / (2nd)$.

Δλ называется **областью свободной дисперсии** (англ. FSR — free spectral range, отсюда ключ `"fsr"` в коде). Смысл названия: если две линии спектра отличаются больше чем на Δλ, пик одной попадёт на соседний пик другой, и прибор их спутает («перекрытие порядков»). При d = 5 мм, λ = 632,8 нм Δλ ≈ 40 пм — очень маленький интервал: интерферометр Фабри — Перо сделан для изучения тонкой структуры очень узких участков спектра.

3.9. Ширина пика, резкость, разрешающая способность



Ширина пика w (полная ширина на половине высоты, англ. FWHM) — расстояние между точками, где T = Tmax/2. По фазе: $1 + F\sin^2(\delta/2) = 2 \;\to\; \sin(\delta/2) = 1/\sqrt{F} \;\to$ полуширина $\delta = 2\arcsin(1/\sqrt{F})$, полная ширина по фазе $\delta_{1/2} = 4\arcsin(1/\sqrt{F})$. Один период по фазе (2π) соответствует Δλ по длине волны, поэтому

$$w = \Delta\lambda\,\frac{\delta_{1/2}}{2\pi}, \qquad \mathcal{F} = \frac{\Delta\lambda}{w} = \frac{2\pi}{\delta_{1/2}} \approx \frac{\pi\sqrt{R}}{1-R} \tag{5}$$

- F — **резкость** (finesse): во сколько раз пик уже расстояния между пиками. Приближение справа получается из $\arcsin x \approx x$ при малом x (большом F); оно хорошо при R > 0,5. При R = 0,9 резкость ≈ 30.
- Ширина существует, только если провалы ниже половины пика: $1/(1+F) < 1/2$, то есть F > 1, R > 0,172. Иначе функция `theory` возвращает `None`, и в таблице прочерк.
- **Разрешающая способность** $A = \lambda/w = m_0 F$: какую относительную разницу длин волн прибор ещё различает. При m_0 ≈ 15 800 и F ≈ 30 получается ≈ 470 000: прибор различает линии, отличающиеся на 1/470 000 длины волны (≈ 1,3 пм).

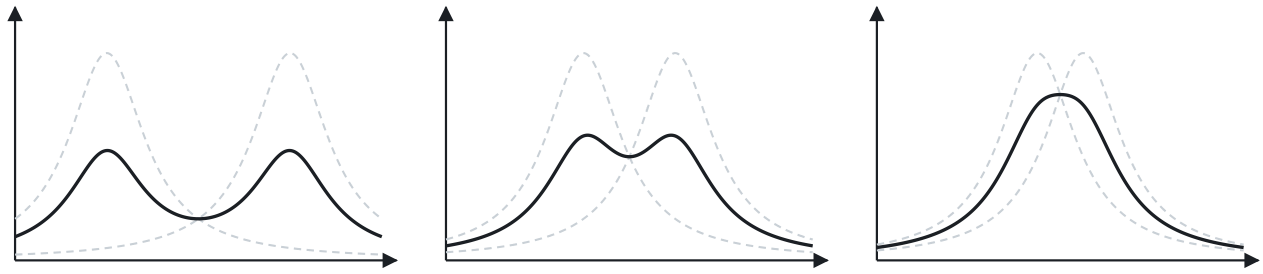
fabry_perot/physics.py — theory

```python
def theory(lam, d, n, R, f, A=0.0):
    F = coefficient_f(R)
    tmax = peak_transmission(R, A)
    fsr = lam ** 2 / (2 * n * d)
    res = {
        "F": F,
        "tmax": tmax,
        "m0": 2 * n * d / lam,
        "T": float(airy(phase(lam, d, n), F, tmax)),
        "fsr": fsr,
        "fsr_nu": C / (2 * n * d),
```

```
        "finesse_approx": math.pi * math.sqrt(R) / (1 - R),
        "contrast": 1 + F,
        "dr2": f * f * n * lam / d,
        "width": None, "finesse": None, "resolving": None,
    }
    if F > 1:
        half = 4 * math.asin(1 / math.sqrt(F))
        res["finesse"] = 2 * math.pi / half
        res["width"] = fsr / res["finesse"]
        res["resolving"] = lam / res["width"]
    return res
```

Критерий разрешения и число 0,81

Две линии одинаковой яркости, разнесённые на δλ, дают сумму двух пиков. Пока пики далеко, между ними глубокий провал. Если сблизить — провал мельчает и исчезает, два пика сливаются в один. Где граница «различимы»? Классический **критерий Рэлея** для дифракционной решётки: максимум одной линии попадает на первый минимум другой; тогда яркость в середине составляет $8/\pi^2 \approx 0{,}81$ от максимума. Для пиков Эйри нет «первого минимума», поэтому программа берёт **ту же глубину провала**: линии различимы, если яркость посередине не больше 0,81 максимума. Эта граница получается при $\delta\lambda \approx 1{,}03 \cdot w$ — почти ровно ширина пика.



δλ = 2w δλ = w δλ = w/2

fabry_perot/physics.py — dip_ratio

```
def dip_ratio(dlam, fsr, F):
    shift = 2 * np.pi * dlam / fsr
    x = np.linspace(-shift, 2 * shift, 6001)
    total = airy(x, F) + airy(x - shift, F)
    middle = float(airy(shift / 2, F) * 2)
    return middle / float(total.max())
```

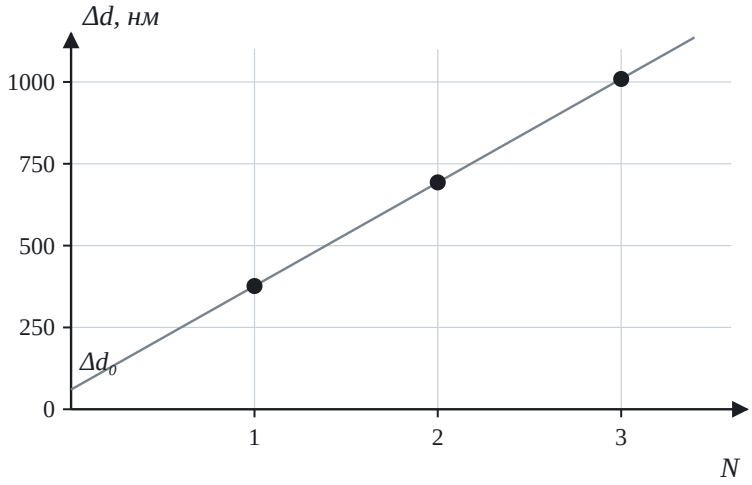
fabry_perot/physics.py — rayleigh_limit

```
def rayleigh_limit(fsr, F):
    lo, hi = 1e-6 * fsr, 0.5 * fsr
    if dip_ratio(hi, fsr, F) > RAYLEIGH:
        return None
    for _ in range(60):
        mid = (lo + hi) / 2
        if dip_ratio(mid, fsr, F) > RAYLEIGH:
            lo = mid
        else:
            hi = mid
    return hi
```

Поиск предела — метод деления пополам (часть 4.6). `dlam % fsr` в `verdict_text` — остаток от деления: при δλ больше Δλ картина повторяется.

3.10. Сдвиг зеркала

Если увеличить зазор на λ/(2n), оптическая толщина 2nd вырастет на λ, порядок в центре m_0 — на 1. В центре «рождается» новое кольцо, а все остальные отодвигаются наружу. Считая новые кольца N при сдвиге Δd, находят длину волны: $\Delta d_N = \Delta d_0 + N\lambda \,/\, (2n)$ — прямая, наклон которой λ/(2n).



fabry_perot/physics.py — seamless_shift

```python
def seamless_shift(lam, n, limit):
    step = lam / (2 * n)
    return max(step, math.floor(limit / step) * step)
```

Анимация «Двигать зеркало» сдвигает Δd по кругу от 0 до наибольшего значения, кратного λ/(2n). Когда сдвиг кратен λ/(2n), картина точно такая же, как в начале, поэтому переход «конец → 0» незаметен.

3.11. Дублет и совпадение колец

Две близкие линии λ и λ + δλ дают две системы колец. При изменении d системы то расходятся, то совпадают. Кольца совпадают, когда в зазоре укладывается ровно на один порядок больше коротких волн: $2nd^* = m\lambda = (m-1)(\lambda+\delta\lambda)$, откуда $d^* = \lambda(\lambda+\delta\lambda) \,/\, (2n\,\delta\lambda) \approx \lambda^2 \,/\, (2n\,\delta\lambda)$. Иначе говоря, при d^* область свободной дисперсии $\Delta\lambda$ как раз равна δλ. Измерив d^*, находят $\delta\lambda = \lambda^2 \,/\, (2nd^*)$ — это задание 6.1.

3.12. Экран и фокус линзы (режим лабораторной)

Линза собирает параллельные лучи, идущие под углом θ, в одну точку своей фокальной плоскости — на расстоянии f от линзы. Если экран стоит на расстоянии L ≠ f, пучок шириной D (в программе 10 мм) даёт на экране не точку, а кружок. Его диаметр находится из подобных треугольников: лучи сходятся в точку на расстоянии f, а на расстоянии L расходятся на D·|L − f|/f. Каждое тонкое кольцо размазывается на ширину этого кружка, и соседние кольца сливаются.

fabry_perot/physics.py — defocus_radius

```python
def defocus_radius(L, f, aperture=APERTURE):
    return aperture / 2 * abs(L - f) / f
```

fabry_perot/physics.py — screen_mean

```python
def screen_mean(r1, r2, lam, d, n, L, F, tmax=1.0):
    s1, s2 = r1 / np.hypot(r1, L), r2 / np.hypot(r2, L)
    return airy_mean(phase(lam, d, n, s1), phase(lam, d, n, s2), F, tmax)
```

Луч через центр линзы не преломляется, поэтому центр кружка лежит в точке r = L·tg θ. Яркость точки экрана — среднее картины по кружку: программа усредняет по отрезку радиусов [r − ρ, r + ρ] через первообразную (часть 4.6). Резче всего кольца при L = f, и только тогда радиусы равны f·tg θ — поэтому перед измерением колец экран ставят в фокус.

3.13. Какая формула где в коде

Формула	Функция	Где показывается
$T = T_{max} / (1 + F \sin^2(\delta/2))$	`physics.airy`	все графики и кольца
$F = 4R/(1-R)^2$; $T_{max} = (1-R-A)^2/(1-R)^2$	`coefficient_f`, `peak_transmission`	таблица
$\delta = 4\pi d\sqrt{(n^2 - \sin^2\theta)}/\lambda$	`phase`	везде
$m_0 = 2nd/\lambda$; m(r)	`theory`, `order_at`	таблица, курсор над кольцами
$\Delta\lambda = \lambda^2/(2nd)$; $\Delta\nu = c/(2nd)$	`theory`	плитка, таблица
w, резкость, разрешающая способность	`theory`	плитки, таблица
радиусы колец	`ring_radii`	таблица «по формуле»
$r_2^2 - r_1^2 = f^2n\lambda/d$	`theory ("dr2")`	таблица
провал 0,81	`dip_ratio`, `rayleigh_limit`	вывод под таблицей
сдвиг λ/(2n)	`seamless_shift`	анимация
расфокусировка D·|L − f|/f	`defocus_radius`, `screen_mean`	кольца в лабораторной

ЧАСТЬ 4. ВЫЧИСЛИТЕЛЬНАЯ МАТЕМАТИКА ПРОГРАММЫ

Как программа сама «меряет» свои графики и почему быстро рисует

4.1. Две колонки таблицы: «по графику» и «по формуле»

«По формуле» — значения из части 3, посчитанные прямо. «По графику» — программа строит кривую из десятков тысяч точек и ищет на ней пики, половину высоты и т. д., как это делал бы человек. Если обе колонки совпадают, значит и формулы, и измерения верны. Значок у плиток показывает расхождение: зелёный меньше 1 %, жёлтый меньше 5 %, красный больше.

4.2. Сколько брать точек

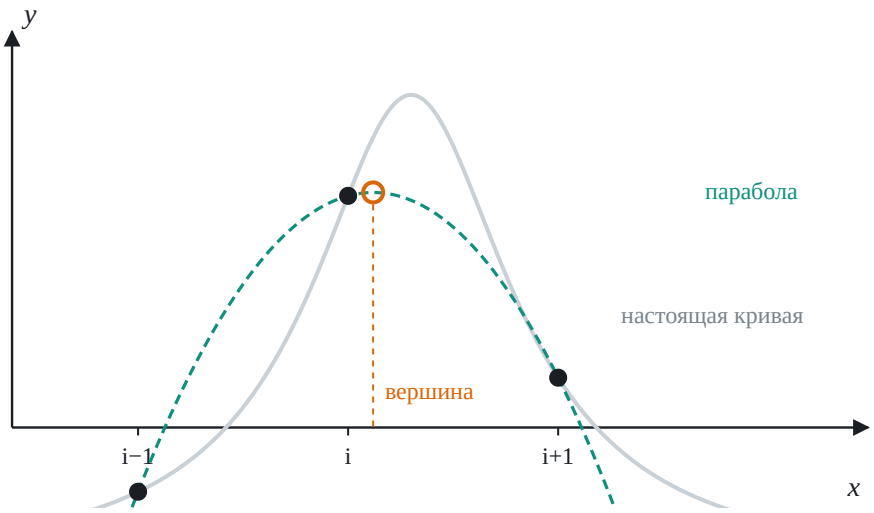
fabry_perot/physics.py — curve_samples

```python
def curve_samples(span, width, fsr):
    return int(min(max(30 * span / (width or fsr), 20000), 2e6))
```

На ширину пика должно прийтись не меньше 30 точек, иначе пик будет измерен грубо. Но не меньше 20 000 (чтобы широкие пики тоже были гладкими) и не больше 2 000 000 (чтобы не тормозить). `width or fsr` — если ширины нет (R слишком мало), берём Δλ.

4.3. Поиск пика и парабола

Самая высокая точка из дискретного набора — не точный пик: настоящая вершина лежит где-то между точками. Уточнение: через три точки (левую, среднюю, правую) проводим параболу и берём её вершину.



Вывод. Пусть точки стоят на x = −1, 0, +1 (в шагах сетки) со значениями L, M, P. Парабола $y = M + bx + cx^2$: из $y(-1) = L$ и $y(1) = P$ получаем $b = (P - L)\,/\,2$, $c = (L - 2M + P)\,/\,2$. Вершина: $x^* = -b\,/\,(2c) = \frac{(L-P)\,/\,2}{L-2M+P}$ — в коде это `shift = 0.5*(left - right)/bend`. Высота вершины $y^* = M + bx^*\,/\,2 = M - \frac{1}{4}(L-P)x^*$ — это `mid - 0.25*(left - right)*shift`. Если `bend = 0` (три точки на прямой), деление запрещено через `np.divide(where=…)`, сдвиг 0.

4.4. Ширина на половине высоты: интерполяция

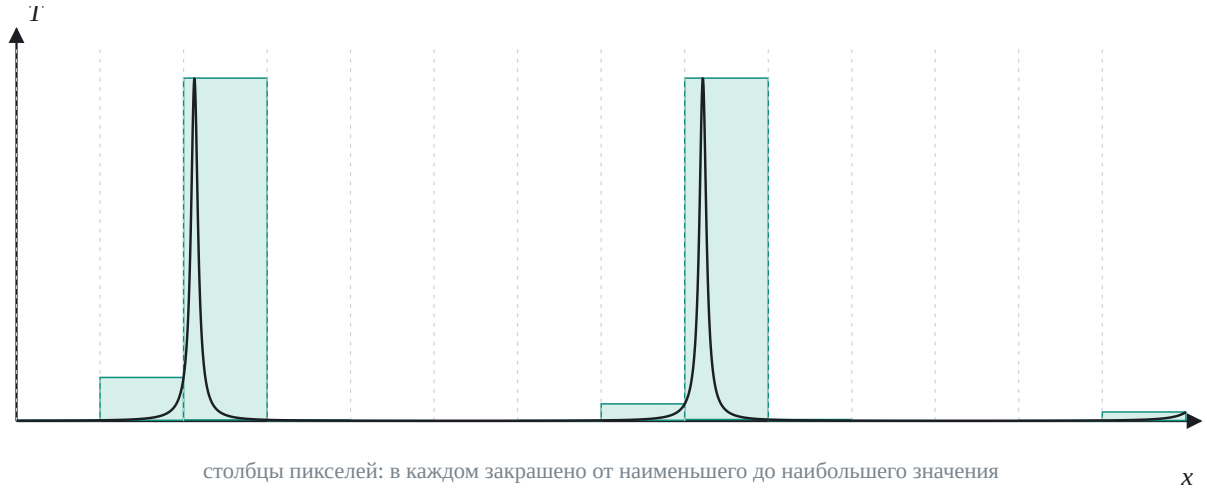
fabry_perot/physics.py — measure_curve

```python
def measure_curve(x, T, x_line):
    res = {"fsr": None, "width": None, "contrast": None, "peaks": None, "half": None}
    if T.size < 3 or T.max() <= 0:
        return res
    idx, pos, height = find_peaks(x, T)
    if len(pos) < 2:
        return res
    k = int(np.clip(np.searchsorted(pos, x_line), 1, len(pos) - 1))
    res["fsr"] = pos[k] - pos[k - 1]
    res["peaks"] = (pos[k - 1], pos[k])
    j = k if abs(pos[k] - x_line) < abs(pos[k - 1] - x_line) else k - 1
    top, i = height[j], idx[j]
    res["contrast"] = top / T.min()
    half = top / 2
    left = np.flatnonzero(T[:i] < half)
    right = np.flatnonzero(T[i:] < half)
    if left.size and right.size:
        a, b = left[-1], i + right[0]
        x_left = np.interp(half, [T[a], T[a + 1]], [x[a], x[a + 1]])
        x_right = np.interp(half, [T[b], T[b - 1]], [x[b], x[b - 1]])
        res["width"] = x_right - x_left
        res["half"] = (x_left, x_right, half)
    return res
```

1. Найти пики, выбрать два соседних вокруг λ_1 (`searchsorted` даёт номер первого пика правее λ_1). `np.clip(…, 1, len(pos) − 1)` не даёт выйти за края списка.
2. $\Delta\lambda$ — разность их положений.
3. Для ближайшего к λ_1 пика: уровень половины `half = top/2`. Слева от пика ищем последнюю точку ниже половины (`left[-1]`), справа — первую (`right[0]`).
4. Между этой точкой и соседней кривая пересекает уровень; место пересечения находим **линейной интерполяцией**: считаем, что между двумя соседними точками кривая — прямая. `np.interp(half, [T[a], T[a+1]], [x[a], x[a+1]])` отвечает на вопрос «при каком x прямая через эти две точки равна half».
5. Контраст — высота пика, делённая на минимум кривой.

4.5. Рисование по столбцам: минимум и максимум на отрезке

> **ПРОСТЫМИ СЛОВАМИ**
>
> Если нарисовать кривую по точкам и соединить их линиями, узкий пик может «проскочить» между точками и пропасть. Программа поступает как цифровой осциллограф: для каждого столбца пикселей находит, где кривая в этом столбце выше и ниже всего, и закрашивает полоску между ними. Тогда ни один пик не потеряется, сколько бы их ни было.



столбцы пикселей: в каждом закрашено от наименьшего до наибольшего значения

fabry_perot/physics.py — airy_range

```python
def airy_range(edges, F, tmax=1.0):
    a, b = np.minimum(edges[:-1], edges[1:]), np.maximum(edges[:-1], edges[1:])
    ta, tb = airy(a, F, tmax), airy(b, F, tmax)
    period = 2 * np.pi
    has_peak = np.floor(b / period) * period >= a
    has_dip = np.floor((b - np.pi) / period) * period + np.pi >= a
    hi = np.where(has_peak, tmax, np.maximum(ta, tb))
    lo = np.where(has_dip, tmax / (1 + F), np.minimum(ta, tb))
    return lo, hi
```

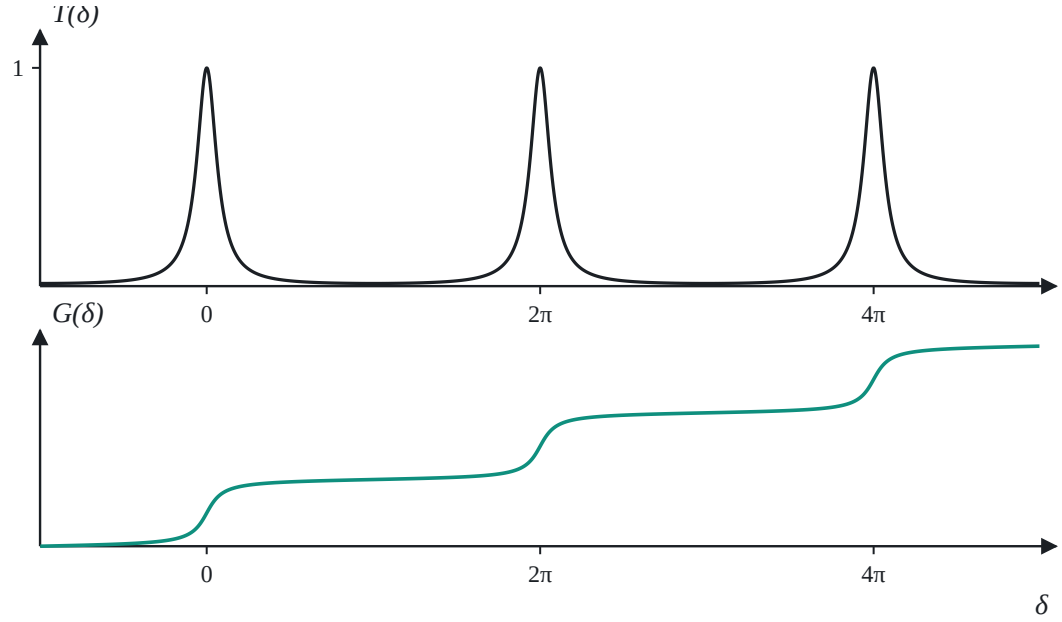
Для функции Эйри минимум и максимум на отрезке фаз [a, b] находятся без перебора. Функция периодична с периодом 2π, пики стоят на 2πm, провалы на π(2m+1). Если внутри отрезка есть число вида 2πm — максимум равен Tmax. Проверка: наибольшее кратное 2π, не превосходящее b, это `floor(b/2π)·2π`; если оно не меньше a — оно внутри отрезка. Аналогично для провалов со сдвигом на π. Если ни пика, ни провала внутри нет, функция на отрезке монотонна, и крайние значения — на концах.

4.6. Средняя яркость пикселя: первообразная

На картинке колец в один пиксель у края экрана могут попасть сотни тонких колец. Если взять яркость в центре пикселя, получится случайная точка на случайном кольце — картина покроется узорами (**муар**). Правильно — взять **среднее** яркости по ширине пикселя. Среднее функции на отрезке — это интеграл, делённый на длину отрезка. У функции Эйри есть первообразная:

$$\int \frac{d\delta}{1 + F\sin^2(\delta\,/\,2)} = \frac{2}{\sqrt{1+F}}\operatorname{arctg}\left(\sqrt{1+F}\,\operatorname{tg}\frac{\delta}{2}\right) \tag{6}$$

(проверяется дифференцированием). Но tg(δ/2) обращается в бесконечность при δ = π, и арктангенс «перескакивает» с +π/2 на −π/2. Поэтому фаза раскладывается на номер периода k и остаток φ ∈ [−π, π): δ = 2πk + φ. За каждый полный период интеграл набирает $2\pi\,/\,\sqrt{1+F}$ («ступенька»), а внутри периода считается по формуле (6).



fabry_perot/physics.py — airy_mean

```python
def airy_mean(d1, d2, F, tmax=1.0):
    q = np.sqrt(1.0 + F)
    k1 = np.floor(d1 / (2 * np.pi) + 0.5)
    k2 = np.floor(d2 / (2 * np.pi) + 0.5)
    a1 = np.arctan(q * np.tan((d1 - 2 * np.pi * k1) / 2))
    a2 = np.arctan(q * np.tan((d2 - 2 * np.pi * k2) / 2))
    width = d2 - d1
    tiny = np.abs(width) < 1e-9
    mean = (2 / q) * (np.pi * (k2 - k1) + (a2 - a1)) / np.where(tiny, 1.0, width)
    return tmax * np.where(tiny, airy((d1 + d2) / 2, F), mean)
```

- `k = floor(δ/2π + 0,5)` — номер периода, в котором $\varphi = \delta - 2\pi k$ попадает в $[-\pi, \pi)$.
- Интеграл от d1 до d2 = $(2/q)\cdot(\pi(k2 - k1) + (\arctg_2 - \arctg_1))$, где $q = \sqrt{(1+F)}$. Делим на длину — получаем среднее.
- Если отрезок почти нулевой, делить нельзя; тогда берётся просто значение в середине. `np.where` выбирает одно из двух для каждого элемента.

Так средняя яркость любого пикселя считается одной формулой, а не суммированием миллионов точек.

4.7. Поиск колец «по картинке»

fabry_perot/physics.py — measure_rings

```python
def measure_rings(lam, d, n, f, r_max, F, tmax, per_period=6, cap=150000, refine=300):
    s2_max = (r_max / math.hypot(r_max, f)) ** 2
    periods = (phase(lam, d, n) - phase(lam, d, n, math.sqrt(s2_max))) / (2 * np.pi)
    count = int(min(max(per_period * periods, 2000), cap))
    u = np.linspace(0.0, s2_max, count)
    du = u[1] - u[0]

    def bright(uu):
        return airy(4.0 * np.pi * d * np.sqrt(n * n - uu) / lam, F, tmax)

    T = bright(u)
    near_u = 2 * du
    near = f * math.sqrt(near_u / (1 - near_u))
    if tmax <= 0:
        return np.zeros(0), near
```

```
    idx = np.flatnonzero((T[1:-1] > T[:-2]) & (T[1:-1] >= T[2:])) + 1
    peaks = u[idx]
    if idx.size:
        center, step = peaks[:refine].copy(), du
        offsets = np.linspace(-1.0, 1.0, 33)
        for _ in range(3):
            grid = np.clip(center[:, None] + step * offsets[None, :], 0.0, s2_max)
            values = bright(grid)
            best = np.argmax(values, axis=1)
            center = grid[np.arange(len(center)), best]
            step = step / 16
        peaks[:refine] = center
    peaks = peaks[(peaks > near_u)]
    return f * np.sqrt(peaks / (1 - peaks)), near
```

- Точки берутся равномерно не по радиусу, а по u = sin²θ. Фаза $\propto \sqrt{n^2 - u}$ почти линейна по u, поэтому на каждое кольцо приходится одинаковое число точек (по 6), хотя кольца к краю сгущаются.
- `periods` — сколько раз фаза проходит 2π от центра до края, то есть сколько колец. Число точек — 6 на кольцо, но от 2000 до 150 000.
- Максимумы находятся как в `find_peaks`, затем первые 300 уточняются «микроскопом»: 3 раза вокруг лучшей точки берутся 33 точки, шаг каждый раз в 16 раз мельче. После трёх шагов точность в $16^3 = 4096$ раз выше шага исходной сетки.
- Кольца ближе к центру, чем 2 шага сетки (`near`), не считаются: это «пятно в центре», а не кольцо. Радиус по u: $\operatorname{tg}\theta = \sqrt{u\,/\,(1-u)}$, $r = f \operatorname{tg}\theta$.

4.8. Метод деления пополам (бисекция)

Задача: найти δλ, при котором провал ровно 0,81. Известно, что при малом δλ провала нет (отношение > 0,81), при большом — есть. Берём отрезок [lo, hi], смотрим середину: если там провала ещё нет, ответ правее — сдвигаем lo в середину, иначе hi. Каждый шаг вдвое сужает отрезок; 60 шагов сужают его в $2^{60} \approx 10^{18}$ раз — точнее, чем хранится число. Метод надёжен: не нужна производная, и он не может «убежать».

4.9. Логарифмическая шкала ползунка

δλ задаётся от 0,01 до 2000 пм — в 200 000 раз. На обычной шкале первое деление ползунка из 1000 было бы уже 2 пм, и малые значения не выставить. На логарифмической шкале равные шаги ползунка — это равные **отношения**: $\delta\lambda = lo \cdot (hi\,/\,lo)^t$, t от 0 до 1. Каждые 200 делений значение растёт в 10 раз.

fabry_perot/params.py — ParamRow.to_slider

```
def to_slider(self, value):
    if self.log:
        t = math.log(value / self.lo) / math.log(self.hi / self.lo)
    else:
        t = (value - self.lo) / (self.hi - self.lo)
    return round(t * self.STEPS)
```

fabry_perot/params.py — ParamRow.from_slider_pos

```
def from_slider_pos(self, pos):
    t = pos / self.STEPS
    return self.lo * (self.hi / self.lo) ** t if self.log else self.lo + t * (self.hi - self.lo)
```

4.10. «Круглые» деления осей

fabry_perot/fmt.py — ticks

```python
def ticks(lo, hi, count=6):
    raw = (hi - lo) / count
    if not raw > 0:
        return [lo], 1.0
    mag = 10 ** math.floor(math.log10(raw))
    step = min((k * mag for k in (1, 2, 5, 10)), key=lambda s: abs(math.log(s / raw)))
    first = math.ceil(lo / step - 1e-9) * step
    total = int(math.floor((hi - first) / step + 1e-9)) + 1
    return [first + i * step for i in range(total)], step
```

1. Желаемый шаг raw = диапазон / 6.
2. Порядок mag = 10^floor(log10(raw)): для raw = 0,037 это 0,01.
3. Из кандидатов 1, 2, 5, 10 × mag выбирается ближайший к raw «в разах» (по модулю логарифма отношения): 0,05.
4. Первое деление — ближайшее кратное шагу не меньше lo; дальше с этим шагом до hi.

`tick_text` пишет подпись с числом знаков после запятой, которое требует шаг: для шага 0,05 — два знака.

ЧАСТЬ 5. ПОГРЕШНОСТИ, МНК И ЛАБОРАТОРНЫЙ РЕЖИМ

5.1. Погрешности измерений: главное

- Любое измерение отличается от истинного значения. **Случайная** погрешность меняется от раза к разу (шум, неточность щелчка), **систематическая** — одинакова всегда (например, неправильная формула).
- Если измерить величину N раз, лучшая оценка — **среднее** $\bar{x} = \frac{1}{N}\sum x_i$.
- Разброс характеризует **выборочное стандартное отклонение** $s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{N-1}}$. Деление на N − 1, а не на N: одна «степень свободы» потрачена на вычисление среднего.
- Среднее точнее одного измерения в √N раз: стандартное отклонение среднего $s \,/\, \sqrt{N}$.
- **Доверительная граница** $\Delta = t \cdot s \,/\, \sqrt{N}$, где t — **коэффициент Стьюдента**. Он показывает, во сколько стандартных отклонений надо отступить, чтобы с вероятностью 0,95 истинное значение попало в интервал $\bar{x} \pm \Delta$. При малом числе измерений t большой (2 измерения — 12,7; 5 — 2,78), при большом стремится к 1,96.
- Результат пишут как $(\bar{x} \pm \Delta)$ единица, погрешность — одной-двумя значащими цифрами, среднее — до того же разряда.

fabry_perot/lab.py — mean_with_error

```python
def mean_with_error(values):
    n = len(values)
    if n == 0:
        return None, None
    m = sum(values) / n
    if n == 1:
        return m, None
    s = math.sqrt(sum((v - m) ** 2 for v in values) / (n - 1))
    return m, student(n - 1) * s / math.sqrt(n)
```

fabry_perot/lab.py — student

```python
def student(dof):
    if dof < 1:
        return float("nan")
    return STUDENT_95[dof - 1] if dof <= len(STUDENT_95) else 1.96
```

5.2. Метод наименьших квадратов (МНК)

> **ПРОСТЫМИ СЛОВАМИ**
>
> Точки на графике почти ложатся на прямую, но не точно. Какая прямая «лучше всех»? Та, у которой сумма квадратов вертикальных отклонений точек от неё наименьшая. Это и есть МНК.

Ищем прямую y = a + bx, минимизирующую $S(a,b) = \sum (y_i - a - bx_i)^2$. В минимуме производные по a и b равны нулю; решение этих двух уравнений:

$$b = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \qquad a = \bar{y} - b\bar{x} \tag{7}$$

Разброс точек вокруг прямой: $s^2 = \frac{\sum (y_i - a - bx_i)^2}{n-2}$ (две степени свободы потрачены на a и b). Отсюда стандартные отклонения коэффициентов:

$$\sigma_b = \frac{s}{\sqrt{\sum (x_i - \bar{x})^2}}, \qquad \sigma_a = s\sqrt{\frac{1}{n} + \frac{\bar{x}^2}{\sum (x_i - \bar{x})^2}} \tag{8}$$

Доверительные границы $\Delta b = t\sigma_b$, $\Delta a = t\sigma_a$ с t для n − 2 степеней свободы.

Прямая через начало координат y = bx (когда теория говорит, что a = 0): $b = \sum x_i y_i \,/\, \sum x_i^2$, $s^2 = \sum (y_i - bx_i)^2 \,/\, (n-1)$, $\sigma_b = s \,/\, \sqrt{\sum x_i^2}$.

fabry_perot/lab.py — fit_line

```
def fit_line(xs, ys, through_origin=False):
    n = len(xs)
    if n < (1 if through_origin else 2):
        return None
    if through_origin:
        sxx = sum(x * x for x in xs)
        if sxx == 0:
            return None
        b = sum(x * y for x, y in zip(xs, ys)) / sxx
        a = 0.0
        dof = n - 1
        s2 = sum((y - b * x) ** 2 for x, y in zip(xs, ys)) / dof if dof > 0 else float("nan")
        sb, sa = math.sqrt(s2 / sxx) if dof > 0 else float("nan"), 0.0
    else:
        mx, my = sum(xs) / n, sum(ys) / n
        sxx = sum((x - mx) ** 2 for x in xs)
        if sxx == 0:
            return None
        b = sum((x - mx) * (y - my) for x, y in zip(xs, ys)) / sxx
        a = my - b * mx
        dof = n - 2
        s2 = sum((y - a - b * x) ** 2 for x, y in zip(xs, ys)) / dof if dof > 0 else float("nan")
        sb = math.sqrt(s2 / sxx) if dof > 0 else float("nan")
        sa = math.sqrt(s2 * (1 / n + mx * mx / sxx)) if dof > 0 else float("nan")
    t = student(dof)
    return {"a": a, "b": b, "sa": sa, "sb": sb, "n": n, "dof": dof, "t": t, "da": t * sa, "db": t
* sb}
```

Функция возвращает словарь, чтобы вызывающий код брал нужное по имени: `fit["b"]`, `fit["db"]`. `float("nan")` — «не число» там, где погрешность не определена (две точки для прямой общего вида: 0 степеней свободы).

5.3. Линеаризация

МНК умеет только прямые, поэтому каждую зависимость переписывают так, чтобы она стала прямой:

Задание	Физика	Ось x	Ось y	Наклон b	Искомое
2. Δλ(d)	$\Delta\lambda = \lambda^2/(2nd)$	1/d	Δλ	$\lambda^2/(2n)$	$\lambda = \sqrt{(2nb)}$
3. w(R)	$w \approx \Delta\lambda(1-R)/(\pi\sqrt{R})$	$(1-R)/\sqrt{R}$	w	$\Delta\lambda/\pi$	$\Delta\lambda = \pi b$
4. кольца	$r^2 = f^2 n\lambda/d\cdot(k-1+\varepsilon)$	k	r^2	$f^2 n\lambda/d$	$\lambda = bd/(f^2 n)$
5. сдвиг	$\Delta d = \Delta d_0 + N\lambda/(2n)$	N	Δd	λ/(2n)	λ = 2nb

fabry_perot/lab.py — derived

```
def derived(key, fit, params, known):
    if fit is None or not fit["b"] or not math.isfinite(fit["b"]):
        return None
    b, db = fit["b"], fit["db"]
    n = params["n"]
```

```
    if key == "fsr":
        lam = math.sqrt(2 * n * b * 1e-15) * 1e9
        return ("λ", lam, lam * db / (2 * b), "нм")
    if key == "width":
        return ("Δλ", math.pi * b, math.pi * db, "пм")
    if key == "rings":
        lam = b * params["d"] / (params["f"] ** 2 * n) * 1e6
        return ("λ", lam, lam * db / b, "нм")
    if key == "shift":
        return ("λ", 2 * n * b, 2 * n * db, "нм")
    return None
```

Единицы: в задании 2 Δλ в пм, 1/d в мм⁻¹, значит b в пм·мм = $10^{-12} \cdot 10^{-3}$ м² = 10^{-15} м²; отсюда множитель `1e-15`, а `* 1e9` переводит метры в нанометры.

5.4. Перенос погрешностей

Если y = f(x), то $\Delta y \approx \left| f'(x) \right| \Delta x$. Частные случаи из программы:

- $\lambda = \sqrt{2nb \cdot 10^{-15}}$ ~ √b: относительная погрешность вдвое меньше: $\Delta\lambda / \lambda = \Delta b / (2b)$ → `lam * db / (2 * b)`.
- λ ~ b: $\Delta\lambda / \lambda = \Delta b / b$ → `lam * db / b`.
- Произведение и частное независимых величин: относительные погрешности складываются квадратично. Для $\delta\lambda = \lambda^2 / (2nd^*)$: $\varepsilon = \sqrt{(2\Delta\lambda / \lambda)^2 + (\Delta d^* / d^*)^2}$ → `math.hypot(2 * dlam_err / lam, e / m)`. Двойка — потому что λ в квадрате.

5.5. Варианты: откуда берутся скрытые значения

fabry_perot/lab.py — Variant.__init__

```
def __init__(self, code):
    self.code = int(code)
    rng = random.Random(f"fabry-perot-variant-{self.code}")
    self.lam = round(rng.uniform(450.0, 690.0), 1)
    self.dlam = float(round(rng.uniform(300.0, 900.0)))
    self.A = rng.choice([0.002, 0.003, 0.004, 0.005, 0.006, 0.008, 0.010])
    rng.choice([100, 120, 150, 180, 200])
    rng.choice([0.02, 0.03, 0.05, 0.08])
    options = [(d, f) for d in (2.0, 3.0, 4.0, 5.0, 6.0) for f in (150.0, 200.0, 250.0, 300.0)]
    rng.shuffle(options)
    for d, f in options:
        if len(ring_radii(self.lam * 1e-9, d * 1e-3, 1.0, f * 1e-3, 0.010)) >= 7:
            break
    self.d_rings, self.f_rings = d, f
```

ПРОСТЫМИ СЛОВАМИ

Компьютер не умеет выдавать по-настоящему случайные числа — он считает их по формуле из начального числа, «зерна» (seed). Одно и то же зерно — одна и та же последовательность. Поэтому вариант 7 даёт одни и те же λ, δλ, A на любом компьютере, в любой день, и преподаватель может заранее посчитать ответы.

- `random.Random("fabry-perot-variant-7")` — свой генератор с зерном-строкой. Строка превращается в число одинаково на всех системах (в отличие от встроенной функции `hash`, которая для строк меняется от запуска к запуску). Свой генератор не зависит от остальной программы: сколько бы раз кто-то ни вызывал `random` в другом месте, варианты не изменятся.

- `round(rng.uniform(450.0, 690.0), 1)` — случайная длина волны с шагом 0,1 нм.
- `rng.choice([...])` — случайный элемент списка; `rng.shuffle(options)` — перемешивание.
- Для задания с кольцами перебираются пары (d, f) в случайном порядке, пока на экране радиусом 10 мм не окажется хотя бы 7 колец. Цикл `for … break`: после `break` переменные `d, f` хранят найденную пару.

5.6. Серии и журнал

Каждое задание — объект `Series`. Поле `kind` говорит, как записывается точка:

- **pair** — два правых щелчка по графику T(λ): y = $|x_2 - x_1|$. Первый щелчок запоминается в `journal.pending`, второй даёт точку. x — функция текущего параметра (`xfunc(ctx[xkey])`, например 1/d).
- **ring** — щелчок по кольцу: y = r^2, x — номер кольца по порядку.
- **param** — кнопка «Записать»: запоминается текущее значение параметра (`source`).
- **point** — просто показание курсора.

Точка хранится словарём `{"x", "y", "note", "ctx"}`, где ctx — параметры окна в момент записи (n, d, f, R). Все серии — словарь списков `self.data`. Он превращается в текст JSON (`json.dumps`) и сохраняется в настройках после каждого изменения; при запуске читается обратно (`load_json`) с проверкой, что каждая запись — словарь с x и y (защита от испорченного файла).

`known_lambda` выбирает среди серий 2, 4, 5 ту, где λ получилась с наименьшей относительной погрешностью, и использует её в заданиях 6 и 7.

Задание 7 (поглощение): из $T_{\max} = (1 - R - A)^2 \,/\, (1 - R)^2$ следует $\sqrt{T_{\max}} = 1 - A \,/\, (1 - R)$, $A = (1 - R)(1 - \sqrt{T_{\max}})$.

ЧАСТЬ 6. QT: КАК УСТРОЕНО ОКНО

Библиотека окон, кнопок и рисования

6.1. Qt, PyQt5 и PyQt6

Qt (читается «кьют») — большая библиотека на языке C++ для программ с окнами. Она одна и та же на Windows, macOS и Linux: программа пишется один раз. **PyQt** — «переходник», через который Qt вызывается из Python. Есть две версии: PyQt5 (для Qt 5) и PyQt6 (для Qt 6). Qt 6 работает только на Windows 10 и новее, поэтому для Windows 7/8/8.1 нужен PyQt5. Различия между ними мелкие, и все они собраны в `qt.py`:

fabry_perot/qt.py — qt

```
import os

QT6 = False
if os.environ.get("FABRY_QT") != "5":
    try:
        from PyQt6 import QtCore, QtGui, QtWidgets
        QT6 = True
    except ImportError:
        pass
if not QT6:
    from PyQt5 import QtCore, QtGui, QtWidgets

Qt = QtCore.Qt
Signal = QtCore.pyqtSignal
QPointF, QRectF, QLineF, QSize = QtCore.QPointF, QtCore.QRectF, QtCore.QLineF, QtCore.QSize
QBrush, QColor, QFont, QIcon, QImage = QtGui.QBrush, QtGui.QColor, QtGui.QFont, QtGui.QIcon,
QtGui.QImage
QLinearGradient, QRadialGradient = QtGui.QLinearGradient, QtGui.QRadialGradient
QPainter, QPalette, QPen, QPixmap, QPolygonF = (QtGui.QPainter, QtGui.QPalette, QtGui.QPen,
                                                QtGui.QPixmap, QtGui.QPolygonF)
QAction = QtGui.QAction if QT6 else QtWidgets.QAction


def event_pos(event):
    return event.position() if QT6 else event.localPos()
```

- В Qt 6 перечисления (наборы констант) пишутся полностью: `Qt.AlignmentFlag.AlignCenter`. В PyQt5 такая запись тоже работает (начиная с 5.11), поэтому в программе везде полная форма — она подходит обоим.
- `QAction` в Qt 6 переехал в модуль QtGui.
- Положение мыши в событии: `event.position()` в Qt 6, `event.localPos()` в Qt 5.
- Остальные модули пишут `from .qt import …` и не знают, какая версия подключена.

Три главных модуля Qt: **QtCore** — основа без графики (таймеры, сигналы, настройки, точки и прямоугольники); **QtGui** — рисование (цвета, шрифты, картинки, художник QPainter); **QtWidgets** — элементы окна (кнопки, поля, таблицы).

6.2. Приложение и цикл событий

> **ПРОСТЫМИ СЛОВАМИ**
>
> Программа с окном не выполняется «сверху вниз до конца». После создания окна она входит в **цикл событий**: бесконечно ждёт, что что-то произойдёт (щелчок, нажатие клавиши, сработал таймер, окну нужно перерисоваться), и вызывает нужную функцию. Потом снова ждёт. Цикл заканчивается, когда закрыли последнее окно.

fabry_perot/app.py — create_app

```
def create_app(argv=None):
    if not QT6:
        QtWidgets.QApplication.setAttribute(Qt.ApplicationAttribute.AA_EnableHighDpiScaling)
        QtWidgets.QApplication.setAttribute(Qt.ApplicationAttribute.AA_UseHighDpiPixmaps)
    try:
        QtGui.QGuiApplication.setHighDpiScaleFactorRoundingPolicy(
            Qt.HighDpiScaleFactorRoundingPolicy.PassThrough)
    except AttributeError:
        pass
    app = QtWidgets.QApplication(sys.argv if argv is None else argv)
    app.setApplicationName("FabryPerot")
    app.setStyle("Fusion")
    font = QFont(app.font())
    font.setFamilies(FONTS)
    app.setFont(font)
    app.setWindowIcon(app_icon())
    QtCore.QLocale.setDefault(QtCore.QLocale(QtCore.QLocale.Language.Russian,
QtCore.QLocale.Country.Russia))
    return app
```

- `QApplication` — объект «программа целиком». Он один на программу и должен быть создан раньше любого виджета.
- `app.exec()` в `main()` — запуск цикла событий. Отсюда управление не вернётся, пока программа работает.
- Отсюда важное правило: **функция, вызванная из цикла, должна быстро закончиться**. Пока она работает, окно не перерисовывается и не реагирует на мышь («зависает»). Поэтому пересчёт сделан быстрым (10 мс), а анимация идёт через таймер, а не через цикл с `sleep`.
- `setStyle("Fusion")` — встроенный в Qt стиль, одинаковый на всех системах; без него кнопки выглядели бы по-разному на Windows и macOS, и наша таблица стилей ложилась бы на них по-разному.
- `font.setFamilies(FONTS)` — список шрифтов по порядку предпочтения: Segoe UI есть на Windows, SF Pro на macOS, Ubuntu/Cantarell/Noto на Linux. Qt берёт первый найденный.
- `QLocale` — «местные правила»: русская локаль делает десятичную запятую в полях ввода.
- Настройки HiDPI — в части 6.10.

6.3. Виджеты

Виджет — любой элемент окна: кнопка, надпись, поле, таблица и само окно. Все они наследники `QWidget`. Виджеты вложены друг в друга: у каждого есть **родитель**; при удалении родителя удаляются и дети. Виджеты, которые использует программа:

Класс	Что это	Где
`QMainWindow`	главное окно со строкой состояния внизу	`MainWindow`
`QWidget`	пустой прямоугольник-контейнер; основа своих виджетов	страницы, Plot, RingView
`QLabel`	надпись или картинка; понимает простой HTML (`<b>`, ` `)	подписи, итог журнала
`QPushButton`	кнопка; `setCheckable(True)` делает её «залипающей»	примеры, шапка
`QSlider`	ползунок с целыми положениями	параметры
`QDoubleSpinBox`, `QSpinBox`	поле ввода дробного и целого числа	параметры, номер варианта
`QCheckBox`	флажок; наш `ToggleSwitch` — его наследник с другим рисунком	переключатели
`QComboBox`	выпадающий список	выбор серии в журнале
`QLineEdit`	поле ввода строки	фамилия, группа
`QTableWidget`	таблица с ячейками `QTableWidgetItem`	результаты, журнал
`QFrame`	виджет с рамкой — основа карточек	`make_card`
`QScrollArea`	прокручиваемая область	левая колонка
`QSplitter`	разделитель, который можно тянуть мышью	три колонки, графики
`QStackedWidget`	стопка страниц, видна одна	«Результаты» / «Журнал»
`QButtonGroup`	группа кнопок, где нажата только одна	примеры
`QDialog`	отдельное окно-вопрос; `exec()` ждёт, пока его закроют, и возвращает «ОК» или «Отмена»	`LabDialog`
`QMessageBox`	готовые окна сообщений и вопросов «Да/Нет»	справка, ошибки
`QFileDialog`	стандартное окно «Сохранить как…»	сохранение файлов

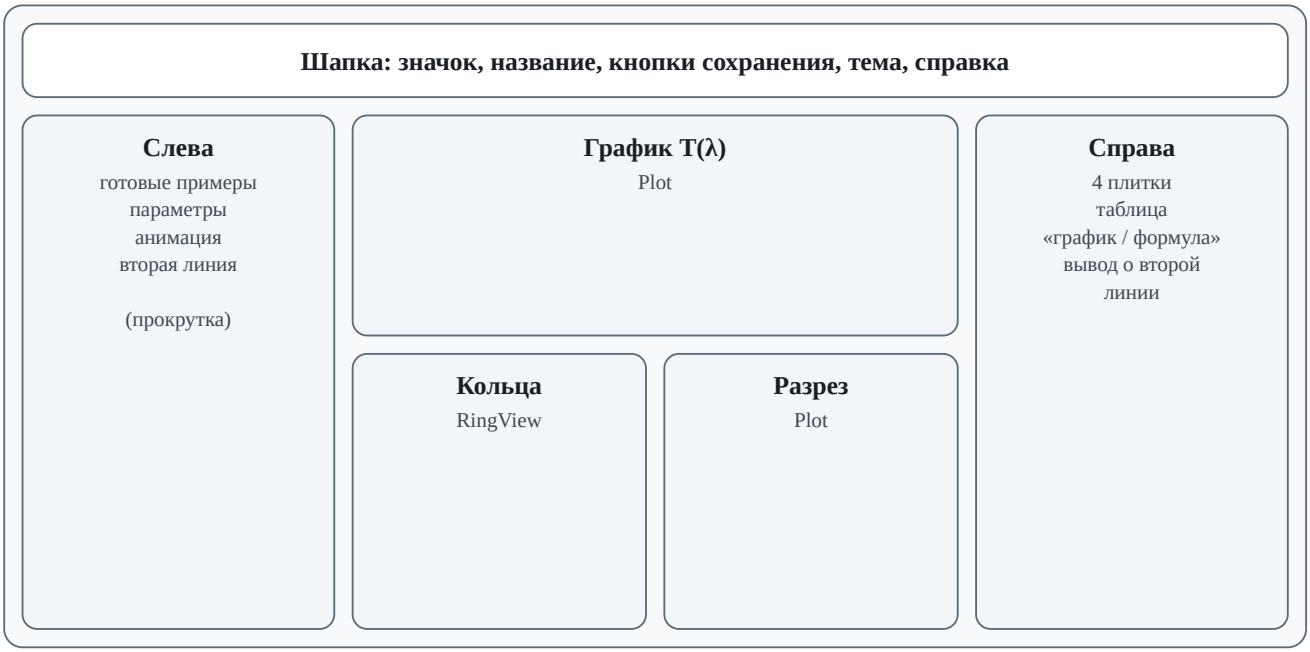
У виджетов есть общие методы: `show(), hide(), setVisible(bool)`; `setEnabled(bool)` (серый и не нажимается); `setToolTip(текст)` (подсказка при наведении); `setObjectName(имя)` (имя для таблицы стилей); `setMinimumSize, setFixedWidth`; `width(), height(), rect()`; `update()` (попросить перерисовать); `setCursor`; `grab()` (снимок в картинку).

6.4. Компоновки (layouts)

> **ПРОСТЫМИ СЛОВАМИ**
> Виджеты не расставляются по координатам вручную. Их кладут в **компоновку**, и она сама расставляет и растягивает их при изменении размера окна.

- `QVBoxLayout` — друг под другом; `QHBoxLayout` — в строку.

- `QGridLayout` — сетка: `grid.addWidget(w, строка, столбец)`. Каждый параметр занимает две строки сетки (сверху название и пределы, снизу ползунок и поле). `setColumnStretch(0, 1)` — первый столбец забирает всё лишнее место.
- `QFormLayout` — «подпись: поле» в две колонки (окно лабораторной).
- `addWidget(w, 1)` — второй аргумент **stretch**: доля лишнего места. `addStretch()` — пустая «пружина», которая отталкивает остальное.
- `setContentsMargins(лево, верх, право, низ)` — поля; `setSpacing` — промежуток между элементами.
- **Политика размера** (`QSizePolicy`): насколько виджет согласен сжиматься и растягиваться. У подзаголовка в шапке стоит `Ignored`: его желаемая ширина не учитывается, и на узком экране он обрезается, а не раздвигает окно.
- Компоновки вкладываются: `layout.addLayout(другая)`.



6.5. Сигналы и слоты

ПРОСТЫМИ СЛОВАМИ

Виджеты сообщают о событиях **сигналами**: «меня нажали», «моё значение изменилось». К сигналу можно **подключить** любую функцию (её называют слотом). Когда сигнал испускается, Qt вызывает все подключённые функции. Кнопка не знает, что делает окно, а окно не опрашивает кнопку — они связаны только подключением.

```
button.clicked.connect(self.save_png)              # щелчок → сохранить снимок
self.slider.valueChanged.connect(self.slider_moved)  # новое положение → функция получит число
self.ring_view.hovered.connect(self.plot_r.set_hover)  # сигнал одного виджета → метод другого
```

Стандартные сигналы, которые использует программа: `clicked` (кнопка), `toggled(bool)` (переключатель), `valueChanged` (ползунок, поле), `currentIndexChanged` (список), `timeout` (таймер), `accepted/rejected` (кнопки диалога), `sectionResized` (ширина столбца таблицы).

Свои сигналы

```
class Plot(QtWidgets.QWidget):
    hovered = Signal(object)          # объявление: сигнал с одним значением любого типа
    picked = Signal(float, float)     # сигнал с двумя дробными числами
    ...
        self.picked.emit(x, y)        # испустить: все подключённые функции получат (x, y)
```

Сигнал объявляется как атрибут класса (`Signal` — это `pyqtSignal`), а испускается методом `emit`. `object` нужен для `hovered`, потому что передаётся либо число, либо `None` (мышь ушла).

Связь курсоров

Разрез колец и картина колец подключены друг к другу «крест-накрест»: наведение на кольца испускает `hovered(радиус)`, разрез ставит свой пунктир на тот же радиус — и наоборот. Зацикливания нет, потому что `set_hover` не испускает сигнал обратно.

`blockSignals`

Ползунок и поле связаны в обе стороны: сдвинул ползунок — поле обновилось; ввёл число — ползунок сдвинулся. Без защиты получилось бы эхо: поле меняет ползунок, ползунок шлёт сигнал и снова меняет поле (с округлением до 1/1000 шкалы!). `slider.blockSignals(True)` временно «глушит» сигналы ползунка на время программной установки.

fabry_perot/params.py — ParamRow.spin_changed

```
def spin_changed(self, value):
    if not self.from_slider:
        self.show_on_slider(value)
    if self.from_slider:
        smooth = False
    elif self.forced is not None:
        smooth = self.forced
    elif self.spin.stepping:
        smooth = "fast"
    else:
        smooth = True
    self.on_change(self.key, smooth)
```

6.6. События

Кроме сигналов, Qt вызывает у виджета методы-события. Переопределив их в своём классе, виджет реагирует по-своему:

Метод	Когда вызывается	Что делает у нас
`paintEvent`	виджет нужно нарисовать (появился, изменился, вызван `update()`)	рисует график, кольца, тумблер
`mouseMoveEvent`	мышь двигается (при `setMouseTracking(True)` — и без нажатой кнопки)	курсор, перетаскивание
`mousePressEvent` / `mouseReleaseEvent`	кнопку мыши нажали / отпустили	правый щелчок — в журнал; левый — начать перетаскивание
`mouseDoubleClickEvent`	двойной щелчок	сброс масштаба
`wheelEvent`	колёсико	масштаб графика, лупа
`leaveEvent`	мышь ушла с виджета	убрать курсор
`resizeEvent`	изменился размер	выбросить готовую картинку
`closeEvent`	окно закрывают	сохранить настройки

- `event.button()` — какая кнопка мыши (`Qt.MouseButton.RightButton`…).
- `event.angleDelta().y()` — поворот колёсика; один «щелчок» колеса = 120 единиц.
- `event.accept()` — «событие обработано, дальше не передавать» (иначе колёсико прокрутило бы ещё и левую колонку).
- `update()` не рисует сразу, а ставит в очередь; несколько вызовов подряд дадут одну перерисовку. Рисовать можно **только** внутри `paintEvent`.

6.7. Таймеры

- `QTimer` испускает сигнал `timeout` через заданный интервал (мс). Обычный — повторяется, `setSingleShot(True)` — один раз. `start()`, `stop()`, `isActive()`.
- `QTimer.singleShot(1500, функция)` — вызвать функцию один раз через 1,5 с (самопроверка).
- `QElapsedTimer` — секундомер: `start()`, потом `elapsed()` — сколько миллисекунд прошло.

В окне три таймера:

1. `timer` (одноразовый, 15 мс) → `recalc`. Ползунок при движении шлёт десятки сигналов; `schedule()` запускает таймер, только если он не запущен. Все сигналы за 15 мс сливаются в один пересчёт.
2. `tween_timer` (16 мс, ≈ 60 кадров в секунду) → `tween_step`, плавные переходы.
3. `scan_timer` (30 мс) → `scan_step`, анимация зеркала.

Таймер срабатывает внутри цикла событий, поэтому между кадрами окно успевает обработать мышь и перерисоваться.

6.8. Настройки между запусками: QSettings

```
s = QtCore.QSettings(QtCore.QSettings.Format.IniFormat, QtCore.QSettings.Scope.UserScope,
                     "SELF-BMSTU", "FabryPerot")
s.setValue("params/lam", 632.8)             # записать
s.value("params/lam")                       # прочитать (придёт строка "632.8" — поэтому
float(...))
s.value("glow", True, type=bool)            # со значением по умолчанию и нужным типом
```

- Формат INI — простой текстовый файл «ключ=значение» с разделами `[params]`. Лежит на Windows в `%APPDATA%\SELF-BMSTU\FabryPerot.ini`, на macOS и Linux в `~/.config/SELF-BMSTU/FabryPerot.ini`. Без `IniFormat` Qt на Windows писал бы в реестр.
- Косая черта в ключе делает раздел: `params/lam` → раздел `[params]`, ключ `lam`.
- `saveGeometry()/restoreGeometry()` — размер и положение окна одной «упаковкой» байтов.
- Значения, прочитанные из файла, проверяются: число приводится к `float` и зажимается в допустимые пределы (`min(max(…, lo), hi)`). Файл мог испортиться или остаться от старой версии.

6.9. Горячие клавиши

fabry_perot/window.py — MainWindow.build_actions

```
def build_actions(self):
    for keys, slot in (("Ctrl+S", self.save_png), ("Ctrl+Shift+S", self.save_rings),
                       ("Ctrl+E", self.save_csv), ("Ctrl+Shift+C", self.copy_table),
                       ("Ctrl+T", self.toggle_theme), ("F1", self.show_help),
                       ("F11", self.toggle_fullscreen), ("Space", self.toggle_scan)):
        action = QAction(self)
        action.setShortcut(QtGui.QKeySequence(keys))
        action.triggered.connect(slot)
        self.addAction(action)
```

`QAction` — «действие», к которому привязывается сочетание клавиш. `QKeySequence("Ctrl+S")` разбирает запись сочетания; на macOS Ctrl автоматически становится ⌘. `self.addAction(action)` — действие работает, пока окно активно.

6.10. Чёткость на экранах с масштабом (HiDPI)

> **ПРОСТЫМИ СЛОВАМИ**
> На экране ноутбука с масштабом 150 % одна «точка» интерфейса — это 1,5 физических пикселя; на Retina (Mac) — 2 пикселя. Если нарисовать картинку по точкам, а экран растянет её в 1,5 раза, она станет мыльной.

- Размеры виджетов в Qt — в логических **точках**. `devicePixelRatioF()` — сколько физических пикселей в точке (1; 1,25; 1,5; 2).
- Поэтому все свои картинки программа создаёт в физических пикселях (ширина × ratio) и сообщает Qt `image.setDevicePixelRatio(ratio)`: «эта картинка плотная, рисуй её в ratio раз мельче». Получается чёткость пиксель в пиксель.
- В Qt 5 это включается вручную атрибутами `AA_EnableHighDpiScaling` и `AA_UseHighDpiPixmaps` до создания приложения; в Qt 6 включено всегда.

- `HighDpiScaleFactorRoundingPolicy.PassThrough` — не округлять 1,25 до 1 или 1,5 до 2, а использовать точный масштаб. Иначе на 125 % всё было бы либо мелким, либо огромным.

6.11. Главное окно целиком

Конструктор `MainWindow.__init__` по шагам:

1. Заводит атрибуты-«состояние»: `shown` (значения, которые сейчас на картинке), `tweens` (идущие переходы), `lab` (вариант или None), секундомер, настройки, три таймера.
2. Собирает страницу: шапка (`build_header`), под ней горизонтальный разделитель из трёх колонок — параметры (`build_controls`), графики (`build_views`), результаты и журнал (`build_results`). `setStretchFactor(1, 1)` — при растяжении окна растёт средняя колонка.
3. Горячие клавиши, номер версии в строке состояния.
4. Размер окна — 92 % × 90 % доступной части экрана (`availableGeometry` — без панели задач).
5. `restore()` — настройки прошлого запуска и первый расчёт. Флаг `ready = True` в конце: пока окно строится, плавных переходов нет, значения ставятся сразу.

ЧАСТЬ 7. ВИЗУАЛ: ЦВЕТА, ТЕМЫ, РИСОВАНИЕ

7.1. Как задаётся цвет

- **RGB** — три числа 0…255: сколько красного, зелёного, синего. Экран смешивает их: (255, 255, 0) — жёлтый.
- **Шестнадцатеричная запись** `#35D0BA`: по две цифры на канал в системе счисления по основанию 16 (0…9, A…F). 35_{16} = 53, $D0_{16}$ = 208, BA_{16} = 186.
- **Альфа** — непрозрачность 0…255: 0 — невидимый, 255 — сплошной. `THEME.color("surface", 235)` — цвет карточки, чуть прозрачный.
- **HSL** — другой способ: оттенок (hue, «какой цвет радуги»), насыщенность (saturation, «насколько не серый»), светлота (lightness). Удобен, когда нужно сделать тот же оттенок светлее или темнее.
- `QColor` — объект цвета Qt; понимает все эти записи: `QColor("#35D0BA")`, `QColor.fromRgbF(r, g, b)` (доли 0…1), `QColor.fromHslF(h, s, l)`; `.name()` — обратно в «#rrggbb».
- **Premultiplied** («предумноженный»): каналы цвета заранее умножены на альфу. Полупрозрачный белый хранится как (128, 128, 128, 128), а не (255, 255, 255, 128). Так Qt накладывает картинки быстрее: не надо умножать при каждом рисовании.

7.2. Две темы

Все цвета собраны в словари `DARK` и `LIGHT` в `theme.py` с одинаковыми ключами: `bg` (фон окна), `surface` (карточки), `raised` (поля), `text`, `muted` (второстепенный текст), `accent` (бирюзовый) и т. д. Тройки `ok/bad/warn/info` — фон, текст и рамка цветных подложек. Смена темы (`toggle_theme`):

1. `THEME.set("light")` — подменить словарь;
2. `apply_theme`: новая палитра Qt, новая таблица стилей, перекрасить радужный ползунок и значки примеров, заново «отполировать» подложки;
3. `recalc()` — перерисовать графики: цвета кривых зависят от фона.

7.3. Таблица стилей (QSS)

> **ПРОСТЫМИ СЛОВАМИ**
> QSS — язык оформления виджетов, почти как CSS веб-страниц. Правило состоит из **селектора** (к каким виджетам применить) и списка **свойств** в фигурных скобках.

```
QLabel#tileValue { font-size: 21px; font-weight: 700; }
QLabel#badge[kind="ok"] { background: #0F2E29; color: #5EE3C9; }
QComboBox:focus { border-color: #35D0BA; }
QSlider::groove:horizontal { height: 6px; border-radius: 3px; background: qlineargradient(...);
}
```

Часть селектора	Значит
`QLabel`	все виджеты этого класса (и его наследников)
`#tileValue`	только с этим именем (`setObjectName("tileValue")`)
`[kind="ok"]`	только если у виджета свойство kind равно "ok" (`setProperty`)
`:focus, :hover, :checked, :disabled`	состояние: в фокусе, под мышью, нажата, выключена
`::groove, ::handle, ::sub-page`	часть сложного виджета: дорожка ползунка, ручка, заполненная часть
A, B	правило для A и для B

Свойства: `background`, `color` (цвет текста), `border` (толщина, стиль, цвет), `border-radius` (скругление), `padding` (внутренний отступ), `margin` (внешний), `font-size`, `font-weight` (700 — жирный), `min-height` и т. п. `qlineargradient(x1:0, y1:0, x2:1, y2:0, stop:0 цвет, stop:1 цвет)` — плавный переход цветов слева направо.

Таблица стилей собирается f-строкой с цветами текущей темы. Фигурные скобки самого QSS в f-строке удваиваются (`{{` и `}}`), иначе Python принял бы их за вставку значения.

Смена вида на лету: `polish`

fabry_perot/widgets.py — polish

```
def polish(widget, kind):
    widget.setProperty("kind", kind)
    widget.style().unpolish(widget)
    widget.style().polish(widget)
```

Qt применяет стиль, когда виджет создаётся. Если потом поменять свойство `kind`, Qt сам этого не заметит. `unpolish/polish` заставляет пересчитать стиль. Так подложка вывода становится зелёной или красной. Чтобы не делать это лишний раз, программа сначала проверяет, изменился ли `kind`.

Палитра

`qt_palette()` — цвета для стандартных окон, которые таблица стилей не покрывает: «Сохранить как…», сообщения, подсказки. `QPalette.ColorRole` — роли: Window (фон), Text, Highlight (выделение) и т. д.

7.4. Цвет длины волны

fabry_perot/theme.py — wavelength_rgb

```
def wavelength_rgb(nm):
    if nm < 440:
        r, g, b = (440 - nm) / 60, 0.0, 1.0
    elif nm < 490:
        r, g, b = 0.0, (nm - 440) / 50, 1.0
    elif nm < 510:
        r, g, b = 0.0, 1.0, (510 - nm) / 20
    elif nm < 580:
        r, g, b = (nm - 510) / 70, 1.0, 0.0
    elif nm < 645:
        r, g, b = 1.0, (645 - nm) / 65, 0.0
    else:
```

```
        r, g, b = 1.0, 0.0, 0.0
    if nm < 420:
        k = 0.3 + 0.7 * (nm - 380) / 40
    elif nm > 700:
        k = 0.3 + 0.7 * (780 - nm) / 80
    else:
        k = 1.0
    return r * k, g * k, b * k
```

Точного перевода «длина волны → RGB» не бывает: монитор не может показать чистые спектральные цвета. Схема Брутона — простое кусочно-линейное приближение: по отрезкам спектра один канал растёт или падает линейно. У краёв спектра (глаз там видит хуже) цвет притемняется множителем k.

`line_color` делает из этого цвет кривой, читаемый на фоне темы: переводит в HSL, на тёмном фоне поднимает светлоту не ниже 0,62 (фиолетовый иначе не виден на чёрном), на светлом — опускает (жёлтый на белом особенно, поэтому `yellowish` — «насколько оттенок близок к жёлтому»). `spectrum_gradient` собирает 21 точку радуги в градиент для дорожки ползунка λ.

7.5. QPainter — художник

Всё своё рисование делается объектом `QPainter`: «художник» рисует на «холсте» — виджете, картинке `QImage` или `QPixmap`.

```
p = QPainter(self)                                  # рисовать на этом виджете
p.setRenderHint(QPainter.RenderHint.Antialiasing)   # сглаживание краёв
p.setPen(QPen(QColor("#FFA54A"), 1.6))              # перо: цвет и толщина линий и текста
p.setBrush(QColor("#111923"))                       # кисть: чем заливать фигуры
p.drawLine(QPointF(0, 0), QPointF(100, 50))
p.drawRoundedRect(QRectF(10, 10, 200, 100), 14, 14) # прямоугольник со скруглением 14
p.drawEllipse(QPointF(50, 50), 4.5, 4.5)            # круг: центр и два радиуса
p.drawText(QRectF(...), Qt.AlignmentFlag.AlignCenter, "текст")
p.drawImage(QPointF(x, y), image)
p.end()                                             # закончить (для QImage обязательно)
```

- **Система координат**: начало в левом верхнем углу, x вправо, **y вниз**. Поэтому в `to_y` большее значение даёт меньший y: `area.bottom() − …`.
- `QPointF, QRectF` — точка и прямоугольник с дробными координатами (F — float). `rect.adjusted(0.5, 0.5, −0.5, −0.5)` — прямоугольник, сжатый на полпикселя: линия толщиной 1 по центру пикселя получается чёткой.
- `Qt.PenStyle.NoPen` — без контура; `Qt.BrushStyle.NoBrush` — без заливки; `pen.setStyle(Qt.PenStyle.DashLine)` — пунктир.
- `p.fontMetrics().horizontalAdvance(текст)` — ширина текста в точках текущим шрифтом; `.height()` — высота строки. Нужны, чтобы подложка была по размеру надписи (`pill`).
- `p.save() / p.translate(x, y) / p.rotate(−90) / p.restore()` — запомнить состояние, сдвинуть начало координат, повернуть, вернуть как было. Так пишется вертикальная подпись оси Y.
- `pixel_font(widget, px, bold)` — шрифт заданного размера в точках экрана (а не в пунктах), чтобы графики выглядели одинаково на всех системах.

7.6. График: координаты, масштаб, кэш

- `area()` — поле графика внутри виджета: отступ 64 точки слева и сверху (подписи и заголовок), 22 справа, 50 снизу.
- `to_x(area, x)` — перевод значения в точку экрана: доля пути от x0 до x1, умноженная на ширину поля. `to_y` — то же по вертикали, перевёрнутое.
- **Масштаб** хранится как `zoom = (a, b)` — доли всей ширины графика, от 0 до 1. `view()` переводит их в значения x. Колёсико (`wheelEvent`) умножает ширину видимой части на 0,8 за щелчок, и точка под курсором остаётся на месте: `center = a + at·(b − a)`, новое `a = center − at·span`. Перетаскивание сдвигает (a, b), не давая выйти за 0 и 1.
- **Кэш.** Рисование сетки, подписей и кривых занимает несколько миллисекунд, а мышь двигается 60–120 раз в секунду. Поэтому готовый рисунок хранится в `QPixmap self.base`. `paintEvent`: если `base` нет — нарисовать его (`draw_base`); потом скопировать его на экран и сверху нарисовать только перекрестие (`draw_overlay`). При новых данных, масштабе или размере `base = None`.

fabry_perot/widgets.py — Plot.paintEvent

```
def paintEvent(self, event):
    ratio = self.devicePixelRatioF()
    if self.base is None:
        self.base = QPixmap(int(self.width() * ratio), int(self.height() * ratio))
        self.base.setDevicePixelRatio(ratio)
        self.base.fill(Qt.GlobalColor.transparent)
        p = QPainter(self.base)
        self.draw_base(p)
        p.end()
    p = QPainter(self)
    p.drawPixmap(0, 0, self.base)
    p.setRenderHint(QPainter.RenderHint.Antialiasing)
    self.draw_overlay(p)
```

7.7. Растеризатор кривой

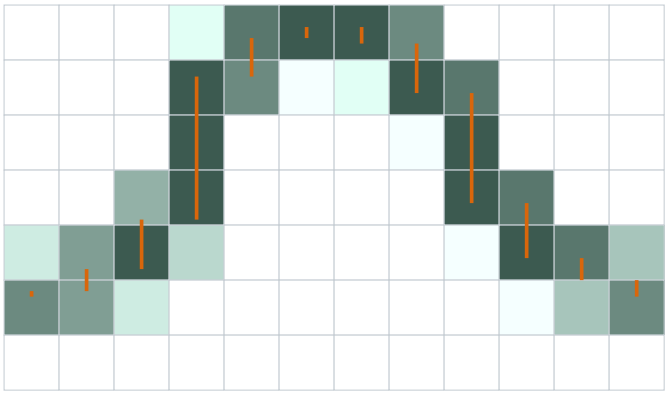
> **ПРОСТЫМИ СЛОВАМИ**
> Растеризация — превращение фигуры в пиксели. Вместо «нарисуй линию через тысячу точек» программа сама решает для каждого пикселя поля графика, насколько он закрашен — сразу для всех пикселей через numpy.

fabry_perot/widgets.py — Plot.draw_curve

```
def draw_curve(self, p, area, curve):
    ratio = self.devicePixelRatioF()
    width = max(int(round(area.width() * ratio)), 1)
    height = max(int(round(area.height() * ratio)), 1)
    x0, x1 = self.view()
    y0, y1 = self.ylim
    lo, hi = curve.span(np.linspace(x0, x1, width + 1))
    scale = height / (y1 - y0)
    top = ((y1 - hi) * scale).astype(np.float32)[None, :]
    bottom = ((y1 - lo) * scale).astype(np.float32)[None, :]
    rows = np.arange(height, dtype=np.float32)[:, None]
    half = np.float32(0.95 * ratio)
    line = np.minimum(rows + 1, bottom + half)
    line -= np.maximum(rows, top - half)
    np.clip(line, 0, 1, out=line)
    fill = rows + 1 - top
```

```
    np.clip(fill, 0, 1, out=fill)
    fade = np.linspace(0.36 if THEME.dark else 0.24, 0.03, height, dtype=np.float32)[:, None]
    fill *= fade
    clear = 1 - line
    clear *= 1 - fill
    if THEME["glow"]:
        dist = np.maximum(top - rows - 0.5, rows + 0.5 - bottom)
        dist /= np.float32(3.2 * ratio)
        np.clip(dist, 0, 1, out=dist)
        glow = 1 - dist
        glow *= glow
        glow *= np.float32(0.3)
        clear *= 1 - glow
    alpha = ((1 - clear) * 255 + 0.5).astype(np.uint8)
    argb = color_table(curve.color)[alpha]
    image = QImage(argb.tobytes(), width, height, 4 * width,
QImage.Format.Format_ARGB32_Premultiplied)
    image.setDevicePixelRatio(ratio)
    p.drawImage(area.topLeft(), image)
```

1. Для каждого из `width` столбцов пикселей `curve.span` даёт наименьшее и наибольшее значение кривой (часть 4.5). Переводим в строки пикселей: `top` и `bottom`.
2. **Линия.** Пиксель строки `row` занимает по вертикали [row, row + 1]. Полоска кривой — [top − half, bottom + half], half ≈ 0,95 пикселя (толщина ~1,9). Доля пикселя внутри полоски — длина пересечения двух отрезков: `min(row + 1, bottom + half) − max(row, top − half)`, обрезанная в [0, 1]. Пиксель, закрытый наполовину, получает прозрачность 0,5 — край линии гладкий (**сглаживание**).



оранжевое — от наименьшего до наибольшего значения в столбце

цвет пикселя — какая его доля попала в полоску (плюс толщина линии)

3. **Заливка** под кривой: доля пикселя ниже верха кривой, умноженная на прозрачность, которая тает книзу (`fade` от 0,36 до 0,03).
4. **Свечение** (в тёмной теме): расстояние от пикселя до полоски, делённое на 3,2 пикселя; чем ближе, тем ярче; `glow *= glow` — квадрат делает спад мягче.
5. **Сложение слоёв**: как наложенные цветные плёнки. Плёнка с непрозрачностью a пропускает долю 1 − a; несколько плёнок пропускают произведение (1 − a)(1 − b)(1 − c). Итоговая непрозрачность 1 − это произведение. Так слои не дают «больше 100 %».
6. Цвет у всей кривой один, поэтому пиксель — это только число 0…255 (прозрачность). Таблица `color_table(color)` из 256 готовых цветов «этот цвет с прозрачностью k» переводит всю картинку в цвета одной операцией `table[alpha]`.

7. Массив чисел `uint32` превращается в картинку Qt: `QImage(байты, ширина, высота, байт_в_строке, формат)`. Байт в строке — 4 × ширина (по 4 байта на пиксель).

fabry_perot/widgets.py — color_table

```
def color_table(color):
    c = QColor(color)
    alpha = np.arange(256, dtype=np.uint32)
    r, g, b = ((alpha * k + 127) // 255 for k in (c.red(), c.green(), c.blue()))
    return (alpha << 24) | (r << 16) | (g << 8) | b
```

`(alpha * k + 127) // 255` — умножение цвета на прозрачность с округлением (premultiplied, часть 7.1); `//` — деление нацело.

7.8. Картина колец

fabry_perot/widgets.py — RingView.make_image

```
def make_image(self, side, scale=None, center=None):
    lines, half, _, (d, n, f), F, tmax, rho = self.data
    scale = self.scale if scale is None else scale
    cx, cy = self.center if center is None else center
    pixel = 2 * half / (side * scale)
    key = (side, scale, cx, cy)
    index = self.index.get(key)
    if index is None:
        c = (np.arange(side) + 0.5 - side / 2) * pixel
        index = (4 * np.hypot(c[None, :] + cx, c[:, None] + cy) / pixel).astype(np.intp)
        self.index = {key: index}
    count = int(index.max()) + 1
    center_r = (np.arange(count) + 0.5) * pixel / 4
    r1, r2 = np.maximum(center_r - pixel / 2 - rho, 0.0), center_r + pixel / 2 + rho
    s1, s2 = r1 / np.hypot(r1, f), r2 / np.hypot(r2, f)
    if self.glow:
        sigma = max(side / 180, 2.0)
        half_k = int(3 * sigma)
        kernel = np.exp(-0.5 * (np.arange(-half_k, half_k + 1) / sigma) ** 2)
        kernel /= kernel.sum()
    rgb = np.zeros((count, 3))
    for lam, color in lines:
        bright = airy_mean(phase(lam, d, n, s1), phase(lam, d, n, s2), F, tmax) / len(lines)
        if self.glow:
            padded = np.pad(bright, half_k, mode="reflect")
            bright = bright + 0.6 * np.convolve(padded, kernel, mode="valid")
        rgb += np.clip(bright, 0, 1)[:, None] ** (1 / 2.2) * np.array(color)
    alpha = np.clip(half / pixel - (np.arange(count) + 0.5) / 4 + 0.5, 0, 1)
    v = (np.clip(rgb, 0, 1) * alpha[:, None] * 255 + 0.5).astype(np.uint32)
    a = (alpha * 255 + 0.5).astype(np.uint32)
    table = (a << 24) | (v[:, 0] << 16) | (v[:, 1] << 8) | v[:, 2]
    argb = table[index]
    return QImage(argb.tobytes(), side, side, 4 * side,
                  QImage.Format.Format_ARGB32_Premultiplied).copy()
```

1. **Цвет зависит только от расстояния до центра**. Поэтому вместо полумиллиона вычислений для пикселей считается одна короткая таблица цветов по расстоянию с шагом ¼ пикселя (несколько тысяч значений), а затем каждый пиксель берёт цвет из неё.
2. **Карта номеров** `index` — для каждого пикселя номер строки таблицы (4 × расстояние / размер пикселя). Она зависит только от размера картинки и положения лупы, поэтому

хранится (`self.index`, словарь с ключом (сторона, увеличение, центр)) и не пересчитывается, пока их не изменят.

3. **Яркость** для каждого расстояния — среднее функции Эйри по ширине пикселя через первообразную (часть 4.6): `airy_mean(p1, p2, …)`, где p1, p2 — фазы на краях окна. Если экран не в фокусе, окно шире на радиус кружка расфокусировки rho с каждой стороны (часть 3.12).
4. **Свечение** — к яркости добавляется её размытая копия: свёртка (`np.convolve`) с **гауссовым ядром** — весами $e^{-x^2 / 2\sigma^2}$, нормированными на сумму 1. Каждая точка становится взвешенным средним соседей: так выглядит ореол на фотографии.
5. **Гамма**. Монитор показывает яркость не пропорционально числу: значение 128 из 255 выглядит не вдвое темнее 255, а примерно в 4,6 раза (яркость ∝ значение^2,2). Чтобы физическая яркость 0,1 выглядела как 0,1, значение возводится в степень 1/2,2. Без этого тусклые кольца слились бы с чёрным фоном.
6. Цвет каждой линии — `wavelength_rgb`; при двух линиях их вклады складываются.
7. **Край экрана**: прозрачность плавно спадает от 1 до 0 на последнем пикселе круга — край гладкий.
8. Числа собираются в 32-битные цвета сдвигами, таблица «раскладывается» по картинке: `table[index]`. `.copy()` в конце — QImage, созданный из байтов, не владеет ими; копия делает свою память, иначе картинка испортилась бы, когда Python освободит временный массив.

Лупа

`scale` — увеличение (1…40), `center` — какая точка экрана прибора (в метрах от центра колец) стоит в середине вида. Размер точки виджета в метрах `pixel = 2·half/(size·scale)`. При повороте колёсика точка под курсором остаётся на месте: `center_new = x − (x − center)·(scale_old/scale_new)`.

Оправа окуляра и подписи

`decorate` один раз рисует прямо на картинке ободок и 72 деления через 5° (каждое шестое — длинное, через 30°). Координаты делений — через cos и sin угла. `paintEvent` поверх картинки рисует подпись, масштабную линейку «круглой» длины (через `ticks`) и пунктирную окружность под курсором со значениями r, m, I.

7.9. Свои мелкие виджеты

- **ToggleSwitch** — наследник `QCheckBox` (значит, у него есть `isChecked` и сигнал `toggled`), но `paintEvent` рисует «таблетку» 32 × 18 и белый кружок слева или справа. `sizeHint` — желаемый размер для компоновки; `hitButton` — щелчок по подписи тоже переключает.
- **make_card** — `QFrame` с именем `card` (оформление из таблицы стилей) и вертикальной компоновкой, в которую кладётся содержимое.
- **pill** — надпись на скруглённой подложке: считает ширину текста, рисует прямоугольник, пишет текст по центру.

- **Значок программы** (`app_icon_image`): скруглённый квадрат с радиальным градиентом (`QRadialGradient` — цвет меняется от центра к краю) и 6 колец радиусом $\propto \sqrt{k}$ (как у настоящей картины) — каждое двумя окружностями: широкой полупрозрачной (ореол) и узкой яркой. `app_icon` собирает значок из 7 размеров: для каждого рисуется вчетверо больший и плавно уменьшается — так мелкие значки не «рваные». `dot_icon` — кружки цветов линий у кнопок примеров.

ЧАСТЬ 8. АНИМАЦИЯ, ПУТЬ ДАННЫХ И СКОРОСТЬ

8.1. Кадры и время

Анимация — это быстрая смена кадров. 60 кадров в секунду — кадр каждые ≈ 16,7 мс. Есть два способа двигать картинку:

- **по кадрам**: каждый кадр прибавлять фиксированный шаг. На медленном компьютере кадров меньше, и анимация идёт медленнее;
- **по времени**: каждый кадр смотреть на секундомер и ставить значение, какое должно быть в этот момент. На медленном компьютере кадров меньше, но переход длится те же 0,85 с. Программа использует этот способ.

8.2. Плавный переход (tween)

Слово tween — от in-between, «промежуточные кадры». Переход описывается пятью числами: откуда, куда, когда начался, по логарифму ли, длительность. Они хранятся в `self.tweens[ключ]`.

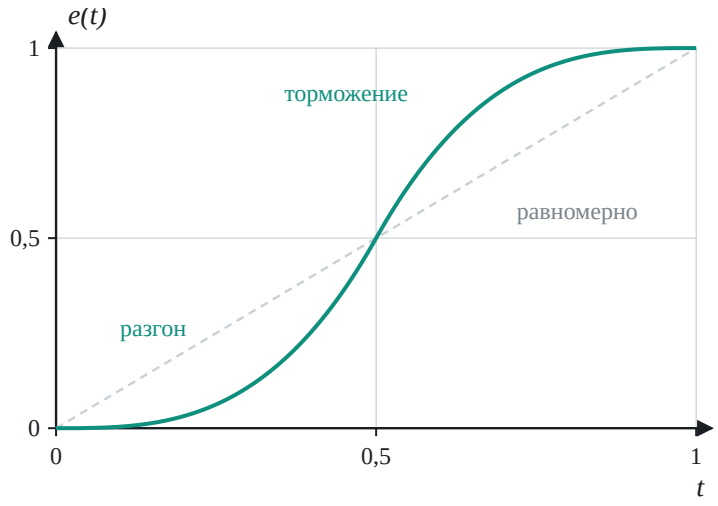
fabry_perot/window.py — MainWindow.tween_step

```
def tween_step(self):
    now = self.clock.elapsed()
    for key, (a, b, start, log, duration) in list(self.tweens.items()):
        t = min((now - start) / duration, 1.0)
        e = ease(t)
        self.shown[key] = a * (b / a) ** e if log and a > 0 and b > 0 else a + (b - a) * e
        if t >= 1.0:
            self.shown[key] = b
            del self.tweens[key]
    if not self.tweens:
        self.tween_timer.stop()
    self.recalc()
```

1. t = (сейчас − начало)/длительность — доля пройденного времени, от 0 до 1.
2. e = ease(t) — доля пройденного пути (см. ниже).
3. Значение: a + (b − a)·e. Для δλ, у которого логарифмическая шкала, путь проходится «по отношениям»: a·(b/a)^e. От 1 пм до 1000 пм на середине пути будет 31,6 пм, а не 500 — движение выглядит равномерным.
4. Когда t дошло до 1, ставится точное конечное значение и переход удаляется. Нет переходов — таймер останавливается.
5. В конце — `recalc()`: картинка по новым промежуточным значениям.

`list(self.tweens.items())` — копия, потому что внутри цикла из словаря удаляются элементы, а менять словарь, по которому идёт цикл, нельзя.

Кривая разгона и торможения



fabry_perot/window.py — ease

```
def ease(t):
    return 4 * t ** 3 if t < 0.5 else 1 - (2 - 2 * t) ** 3 / 2
```

При равномерном движении (e = t) картинка резко трогается и резко встаёт — это выглядит механически. Кубическая кривая ease-in-out: первая половина — $4t^3$ (медленный старт, ускорение), вторая — зеркальная $1 - (2 - 2t)^3 / 2$ (торможение). В точке t = 0,5 обе дают 0,5, и скорости совпадают — стык незаметен. Скорость в начале и в конце нулевая.

8.3. Откуда пришло значение: ползунок, ввод или стрелка

Задача: ползунок меняет картину сразу, введённое число — плавно (0,85 с), шаг стрелкой или колёсиком — коротким переходом (0,15 с), готовый пример — плавно. Всё это приходит одним сигналом `valueChanged` поля. Различаются флажками:

- `from_slider` — ставит `slider_moved` на время установки значения в поле → «сразу».
- `forced` — ставит программа в `set(value, smooth)` (примеры, восстановление) → как велено.
- `spin.stepping` — ставит `StepSpin.stepBy` (стрелки и колёсико) → «быстро».
- ничего из этого → число ввели с клавиатуры → плавно.

`setKeyboardTracking(False)`: поле присылает значение только после Enter или ухода из поля, а не после каждой цифры. Иначе при вводе «12» картина поплыла бы сначала к 1, потом к 12.

8.4. Изменение параметра

fabry_perot/window.py — MainWindow.param_changed

```
def param_changed(self, key, smooth):
    if key not in self.params:
        return
    target = self.params[key].value()
    if key == "dd" and self.scan_timer.isActive():
        self.scan_timer.stop()
        self.update_scan_button()
```

```
    if smooth and self.ready and key in self.shown:
        duration = FAST_MS if smooth == "fast" else TWEEN_MS
        self.tweens[key] = (self.shown[key], target, self.clock.elapsed(), self.params[key].log,
duration)
        if not self.tween_timer.isActive():
            self.tween_timer.start()
    else:
        self.tweens.pop(key, None)
        self.shown[key] = target
        self.schedule()
    if not self.applying:
        self.check_preset(None)
```

- `self.shown` — значения, которые сейчас на картинке; `params[key].value()` — что в поле. Во время перехода они отличаются.
- Переход начинается **с того, что сейчас на картинке**. Если ввести новое число, пока идёт старый переход, картинка не прыгнет — новый переход подхватит движение с текущего места.
- Если параметр меняли руками (не пример), подсветка кнопки примера снимается.

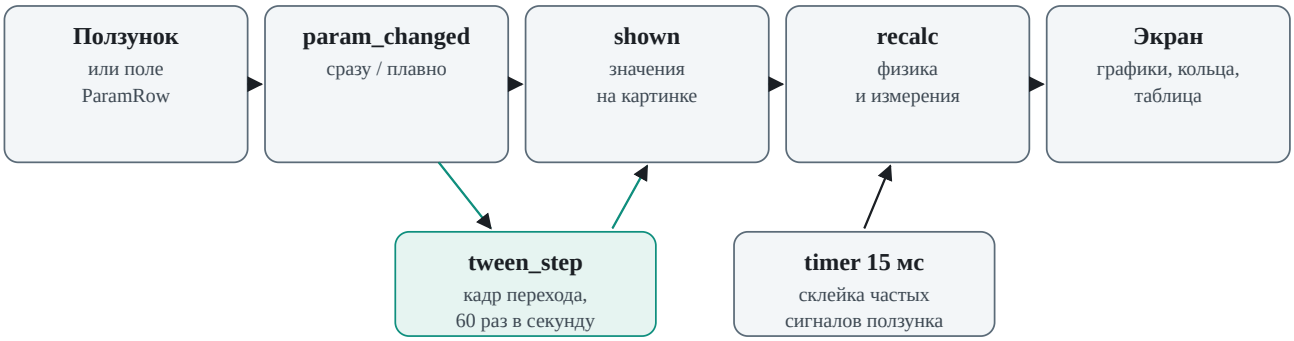
8.5. Анимация зеркала

fabry_perot/window.py — MainWindow.scan_step

```
def scan_step(self):
    lam, n = self.shown["lam"], self.shown["n"]
    end = seamless_shift(lam, n, self.params["dd"].hi)
    value = self.shown["dd"] + lam / (2 * n) / 56
    if value > end:
        value -= end
    self.shown["dd"] = value
    self.params["dd"].display(value)
    self.recalc()
```

Каждые 30 мс Δd растёт на λ/(2n)/56: одно новое кольцо за 56 кадров ≈ 1,7 с. При переходе за `seamless_shift` значение начинается заново без скачка (часть 3.10). `display` показывает значение в поле и на ползунке, заглушив сигналы, — иначе поле сообщило бы об изменении и запустило бы ещё и переход.

8.6. Путь данных: от ползунка до картинки



Главная функция — `recalc`. По шагам:

1. `effective()` — значения для физики: видимые, а в лабораторной вместо λ и A — скрытые из варианта.

2. Перевод в СИ: нм → м (`* 1e-9`), мм → м; d = d + Δd. Поглощение ограничивается: A ≤ 1 − R.
3. `theory(…)` — всё «по формуле». В лабораторной — расстояние до экрана L и радиус кружка расфокусировки `defocus_radius`.
4. График T(λ): десятки тысяч точек вокруг λ_1 (±1,5 Δλ), измерения `measure_curve`, затем `show_transmission` отдаёт графику кривую-формулу, метки λ_1, λ_2 и отрезки Δλ, w.
5. Картина колец: `ring_view.show_data` — сама картинка посчитается при рисовании.
6. Радиусы колец «по картинке» (`measure_rings`) и «по формуле» (`ring_radii`).
7. Разрез колец: кривые-формулы для каждой линии (две линии делят свет пополам) и кривая суммы.
8. Таблица и плитки (`fill_table`), вывод о разрешении (`verdict_text`).

Виджеты при этом ничего не рисуют сразу: `show_data` только запоминает данные и вызывает `update()`. Рисование произойдёт, когда Qt вызовет `paintEvent`.

8.7. Почему быстро

Приём	Что экономит
Кривые — формулы, а не точки; рисование по столбцам через min/max на отрезке	время кадра не зависит от числа пиков: 1000 пиков рисуются так же быстро, как 3
Растеризация numpy одной картинкой вместо тысяч вызовов drawLine	десятки миллисекунд на кадр
Яркость пикселя колец через первообразную	одна формула вместо суммирования миллионов точек
Таблица цветов по расстоянию + карта номеров пикселей	несколько тысяч вычислений вместо полумиллиона
float32 при растеризации, операции «на месте»	память и время на копирование
Кэш готового рисунка; мышь перерисовывает только курсор	движение мыши почти ничего не стоит
Сливание сигналов за 15 мс в один пересчёт	ползунок не вызывает 100 пересчётов в секунду
Высота строк таблицы пересчитывается, только когда картина стоит	Qt не перемеряет текст каждый кадр
Анимация по времени, а не по кадрам	на слабом компьютере меньше кадров, но та же длительность

Как измеряется скорость: засекается время до и после функции (`time.perf_counter()`) или запускается профилировщик `python -m cProfile -s cumtime run.py`, который показывает, в каких функциях программа провела больше всего времени. Так были найдены узкие места первой версии (до 350 мс на кадр).

ЧАСТЬ 9. СОХРАНЕНИЕ

9.1. Снимок окна и картинка колец

- `QFileDialog.getSaveFileName(окно, заголовок, имя_по_умолчанию, фильтр)` — стандартное окно «Сохранить как». Возвращает пару (путь, выбранный фильтр); если нажали «Отмена», путь пустой, и `if path:` ничего не делает. `path, _ = …` — второе значение не нужно.
- `self.centralWidget().grab()` — снимок всего содержимого окна в `QPixmap`; `.save(path, "PNG")` возвращает True или False.
- `write_rings` создаёт картинку 2048 × 2048, заливает тёмным фоном и рисует на ней ту же `make_image`, что и на экране, только большего размера и без лупы. Формат PNG сжимает без потерь — кольца остаются чёткими.

9.2. CSV для Excel

CSV (comma-separated values) — таблица в виде текста: строки файла — строки таблицы, значения разделены разделителем. Русский Excel ожидает разделитель «;» (потому что запятая занята под десятичную дробь) и десятичную запятую. Поэтому `csv.writer(file, delimiter=";")`, числа через `fmt.plain` (запятая, 6 значащих цифр) и кодировка `utf-8-sig` (часть 1.8). `writerow([])` — пустая строка между разделами. Точек графика может быть до 2 миллионов, поэтому пишется каждая `step`-я, не больше ≈ 20 000 строк: `x[::step]` — срез с шагом.

В лабораторной сохраняется только журнал и видимые параметры (`write_lab_csv`), иначе в файле оказалась бы скрытая длина волны.

9.3. Буфер обмена

`QApplication.clipboard().setText(текст)`. Столбцы разделены табуляцией `\t`, строки — переводом строки `\n`. Excel и Word при вставке такого текста сами делают таблицу.

9.4. Что и где хранится между запусками

Ключ в настройках	Что	Когда пишется
`theme, glow, second`	тема, свечение, вторая линия	при закрытии
`params/…`	значения всех параметров	при закрытии
`geometry`	размер и положение окна	при закрытии
`journal`	весь журнал в JSON	после каждой записи
`lab/code, lab/name, lab/group, lab/doublet`	незавершённая лабораторная	при начале и конце работы и при закрытии

JSON — текстовый формат данных, похожий на запись словарей и списков Python: `{"fsr": [{"x": 1.0, "y": 40.04, …}], …}`. `ensure_ascii=False` оставляет русские

буквы как есть, а не в виде `p…`.

ЧАСТЬ 10. АВТОМАТИЧЕСКИЕ ПРОВЕРКИ

10.1. Зачем тесты

> **ПРОСТЫМИ СЛОВАМИ**
> Тест — маленькая программа, которая запускает кусок основной программы и проверяет результат. После любого изменения достаточно одной команды, чтобы узнать, не сломалось ли что-то. Все 29 тестов проходят за несколько секунд; GitHub запускает их перед каждой сборкой, и сломанная программа не соберётся.

10.2. pytest

pytest — библиотека для тестов. Она находит в папке `tests` файлы `test_*.py`, в них — функции `test_*`, выполняет каждую и сообщает, какие упали.

```
python -m pytest tests             # все тесты
python -m pytest tests -v          # с именем каждого теста
python -m pytest tests -k rings    # только тесты, в имени которых есть «rings»
python -m pytest tests -x          # остановиться на первой ошибке
```

- **`assert условие`** — команда Python: если условие ложно, возникает ошибка. pytest показывает, какие значения были слева и справа.
- **`pytest.approx`** — сравнение дробных чисел с допуском: `got == pytest.approx(expect, rel=1e-3)` — отличие не больше 0,1 %.
- **`np.allclose(a, b, rtol, atol)`** — то же для массивов numpy.
- **`@pytest.mark.parametrize("R", [0.3, 0.7, 0.9, 0.98])`** — запустить тест четыре раза с разными R.
- **`pytest.importorskip("numpy")`** — если библиотеки нет, тест пропускается, а не падает.

10.3. Фикстура: окно для тестов

```
@pytest.fixture(scope="module")
def window(tmp_path_factory):
    home = tmp_path_factory.mktemp("settings")
    QtCore.QSettings.setPath(QtCore.QSettings.Format.IniFormat,
                             QtCore.QSettings.Scope.UserScope, str(home))
    ...
    win = MainWindow()
    win.show()
    yield win                  # тесты получают окно
    win.close()                # после всех тестов модуля — закрыть
```

- **Фикстура** — функция, которая готовит то, что нужно тестам. Тест просит её, называя аргумент тем же именем: `def test_slider_is_instant(window)`.
- `scope="module"` — окно создаётся один раз на весь файл, а не для каждого теста.
- `yield` — «отдать значение и подождать»: код после него выполнится, когда тесты закончатся.
- `tmp_path_factory.mktemp` — временная папка; `QSettings.setPath` направляет настройки туда. Тесты не видят и не портят настройки настоящей программы, и каждый запуск начинается с чистого листа.

- Переменная `QT_QPA_PLATFORM=offscreen` (ставится в начале файла через `os.environ.setdefault`) — Qt рисует окна в памяти. Так тесты идут на сервере без монитора.

```
def wait(ms):
    loop = QtCore.QEventLoop()
    QtCore.QTimer.singleShot(ms, loop.quit)
    loop.exec()
```

Как подождать, чтобы при этом работали таймеры окна (плавный переход)? `time.sleep` не подходит: он остановил бы всё. Функция запускает маленький цикл событий и выходит из него через ms миллисекунд — за это время таймеры окна успевают сработать.

10.4. Что проверяется

Файл	Проверки
`test_physics.py`	формула Эйри совпадает с прямым сложением 800 лучей; измерения по графику совпадают с формулами при разных R; поглощение; нет света при R + A = 1; радиусы колец; сдвиг без скачка; первообразная совпадает с плотным усреднением; min/max на отрезках правильные; кольца «по картинке» совпадают с формулой; 22 000 тонких колец не теряются
`test_lab.py`	варианты одинаковы при повторном создании и различны между собой; МНК совпадает с `numpy.polyfit`; по идеальным сериям находится верная λ; реализм не сдвигает пики; предел по провалу ≈ w; среднее с погрешностью
`test_app.py`	окно запускается; введённое число плавно переходит (на полпути значение промежуточное); ползунок меняет сразу; примеры и тема; анимация; сохранение PNG и CSV; короткий переход стрелкой; лабораторная скрывает λ и пишет в журнал; реализм снижает пики; лупа и кривая суммы

Каждый тест устроен одинаково: подготовить → выполнить → сравнить с тем, что должно быть. Например, «введённое число плавно переходит»: поставить в поле d = 10, сразу проверить, что картинка ещё старая, через 350 мс — что значение между старым и 10, через ещё 900 мс — что ровно 10.

ЧАСТЬ 11. УПАКОВКА: ИЗ КОДА — ПРОГРАММА

Как получаются файлы, которые запускаются двойным щелчком без установленного Python

11.1. Задача

Чтобы запустить `run.py`, нужны Python нужной версии, numpy, PyQt и сама библиотека Qt (десятки мегабайт файлов на C++). У студента на компьютере в аудитории этого нет. **Упаковка** — сложить всё это в один файл или одну папку, которая запускается сама.

11.2. PyInstaller

> **ПРОСТЫМИ СЛОВАМИ**
>
> PyInstaller читает `run.py`, находит все импорты (и импорты импортов), копирует сам Python, все нужные модули и библиотеки в одно место и добавляет маленькую программу-«загрузчик» (bootloader), которая при запуске поднимает этот встроенный Python и выполняет `run.py`.

Два режима:

	Один файл (`--onefile`)	**Папка (по умолчанию; наш portable)**
Что получается	один `FabryPerot.exe` ≈ 40 МБ	папка `FabryPerot/`: exe и сотни файлов рядом
Как запускается	при **каждом** запуске распаковывает всё содержимое во временную папку (`%TEMP%\_MEIxxxxx`), запускает оттуда, при выходе удаляет	запускается сразу: всё уже лежит рядом
Старт	1–3 с на SSD, 5–10 с на старом жёстком диске	≈ 1 с
Скорость работы	одинаковая: после запуска работает один и тот же код	
Удобство	один файл легко переслать	нужно распаковать zip, не разбрасывать файлы

Слово **portable** («переносная») значит: программа не требует установки, её можно положить на флешку и запустить где угодно; она ничего не пишет в систему (кроме файла настроек).

Ключи PyInstaller из нашей сборки

Ключ	Значение
`--noconfirm`	не спрашивать, можно ли стереть прошлую сборку
`--onefile`	один файл (без него — папка)
`--windowed`	программа с окном: на Windows не открывать чёрное окно консоли, на macOS собрать `.app`
`--name FabryPerot`	имя exe и папки
`--icon assets/icon.ico`	значок файла (на macOS — `.icns`)
`--version-file …`	сведения «Свойства → Подробно» у exe (Windows)
`--add-binary "папка\*.dll;."`	положить внутрь дополнительные библиотеки; после «;» — куда внутри (точка — в корень)
`--exclude-module tkinter`	не класть модуль, даже если он найдётся (tkinter и вторая версия PyQt не нужны, а весят много)
`--distpath, --workpath`	куда класть результат и промежуточные файлы (чтобы две сборки не мешали друг другу)
`--osx-bundle-identifier`	уникальное имя приложения для macOS (`ru.bmstu.self.fabryperot`)

Собирать под систему можно только **на этой же системе**: exe для Windows — на Windows, `.app` — на Mac. Поэтому сборку делают три разных компьютера GitHub (часть 12).

11.3. Windows 7–11

Почему именно такие версии

Что	Версия	Почему
Python	3.8	последняя версия Python, которая запускается на Windows 7 (3.9 и новее — только Windows 8.1+)
Разрядность	32 бита (x86)	32-битная программа запускается и на 32-, и на 64-битной Windows (64-битная Windows запускает 32-битные программы через встроенную подсистему WOW64). 64-битная программа на 32-битной Windows не запустится. Программе не нужно больше 2 ГБ памяти, так что 32 бит достаточно
PyQt5 / Qt	5.15	Qt 6 требует Windows 10; Qt 5.15 — последняя версия с Windows 7
numpy	1.24.4	последняя версия для Python 3.8
PyInstaller	5.13.2	версия, у которой загрузчик и набор функций рассчитаны на Python 3.8 и Windows 7

Зависимости, которые ставятся на сборочном компьютере, перечислены в `requirements/win7.txt` со знаком `==` — ровно эта версия. Это называется «закрепить версии»: завтра выйдет новая версия библиотеки, и сборка от этого не сломается.

UCRT

Python 3.8 написан на C и использует стандартную библиотеку C от Microsoft — **Universal C Runtime** (файлы `ucrtbase.dll` и `api-ms-win-crt-*.dll`). В Windows 10 и 11 она встроена. В Windows 7 её приносит обновление KB2999226, которое на компьютерах в

аудиториях часто не установлено. Сборка находит эти файлы в Windows SDK на сборочном компьютере (папка `Windows Kits\10\Redist\…\ucrt\DLLs\x86`) и кладёт внутрь exe ключом `--add-binary`. Кусок сборки на PowerShell:

```
$ucrt = Get-ChildItem "${env:ProgramFiles(x86)}\Windows Kits\10\Redist" -Recurse -Directory |
        Where-Object { $_.FullName -match 'ucrt\\DLLs\\x86$' } | Select-Object -First 1
```

- PowerShell — командная оболочка Windows. Переменные начинаются с `$`.
- `Get-ChildItem … -Recurse -Directory` — все папки внутри, на любой глубине.
- `|` — «передать результат следующей команде» (конвейер).
- `Where-Object { … -match '…' }` — оставить те, чей путь подходит под регулярное выражение (`\\` — обратная косая, `$` — конец строки).
- `Select-Object -First 1` — первую найденную.
- `@('a', 'b')` — список; `pyinstaller @common` — подставить все элементы списка как аргументы.

Сведения о версии

`packaging/windows/version.txt` — описание в формате PyInstaller (это код Python, который PyInstaller выполняет): версия 2.0.0, название, организация. `041904B0` и `[0x0419, 1200]` — код языка (0x0419 — русский) и кодовой страницы (1200 = Unicode). Эти сведения видны в «Свойствах» файла и в диспетчере задач.

SmartScreen

Windows предупреждает о программах без цифровой подписи от известного издателя. Подпись стоит денег (сертификат разработчика), поэтому её нет. «Подробнее → Выполнить в любом случае» — обычный путь для учебных программ.

11.4. macOS

Что такое .app

Программа на Mac — это папка с окончанием `.app`, которую Finder показывает как один значок:

```
Fabry-Perot.app/
  Contents/
    Info.plist          сведения о программе: имя, версия, значок, минимальная версия macOS
    MacOS/Fabry-Perot   сама программа (загрузчик PyInstaller)
    Resources/          значок .icns
    Frameworks/         Python, Qt, numpy
```

`Info.plist` — файл в формате XML со списком «ключ — значение». `PlistBuddy` — встроенная в macOS программа для его правки: `Add :CFBundleDisplayName string 'Фабри — Перо'` добавляет русское имя, `Set :CFBundleShortVersionString 2.0.0` — версию, `LSMinimumSystemVersion 11.0` — «не ниже macOS 11». `|| true` в конце команды оболочки — «если команда не удалась (ключ уже есть), не считать это ошибкой».

Две сборки: Apple Silicon и Intel

Маки бывают на процессорах Apple (M1–M4, архитектура arm64) и на Intel (x86_64). Это разные системы команд процессора, поэтому программа собирается дважды на двух разных сборочных компьютерах: `macos-14` (Apple Silicon) и `macos-15-intel`. Переменная `MACOSX_DEPLOYMENT_TARGET=11.0` говорит компилятору «делай совместимо с macOS 11». numpy закреплён на 1.26.4: его готовые сборки работают с macOS 11, а у более новых — только с macOS 14.

Подпись и Gatekeeper

На Apple Silicon программа обязана быть подписана, хотя бы «ad-hoc» (без сертификата разработчика): `codesign --force --deep --sign - Fabry-Perot.app`. Дефис вместо имени сертификата — и есть ad-hoc; `--deep` — подписать и все библиотеки внутри. **Gatekeeper** (защита macOS) всё равно предупредит о программе из интернета от неизвестного разработчика: поэтому первый запуск — правый щелчок → «Открыть». Если macOS пишет «повреждена», команда `xattr -cr Fabry-Perot.app` снимает с файлов метку «скачано из интернета» (расширенный атрибут `com.apple.quarantine`).

Образ .dmg

```
mkdir dmg
cp -R "dist/Fabry-Perot.app" dmg/
ln -s /Applications dmg/Applications          # ярлык на «Программы», чтобы перетащить
hdiutil create -volname "Fabry-Perot" -srcfolder dmg -ov -format UDZO FabryPerot-macOS-
Intel.dmg
```

DMG — файл образа диска. При открытии он появляется как диск, внутри — программа и ярлык на «Программы». `ln -s` создаёт символическую ссылку (ярлык). `hdiutil` — встроенная программа для образов; `UDZO` — сжатый образ только для чтения; `-ov` — перезаписать, если есть.

11.5. Linux

- **glibc.** Почти все программы Linux используют системную библиотеку C (glibc). Программа, собранная на системе с новой glibc, не запустится на старой, а наоборот — запустится. Поэтому сборка делается на Ubuntu 22.04 (glibc 2.35): она работает на Ubuntu 22.04 и новее, Debian 12, Fedora 36+.
- **xcb.** Окна в Linux рисует графический сервер X11 (или Wayland с совместимостью с X11). Qt общается с ним через библиотеки `libxcb-*`. На сборочном компьютере они ставятся через `sudo apt-get install` (`apt` — установщик пакетов Ubuntu, `sudo` — от имени администратора), и PyInstaller кладёт их внутрь файла — на компьютере пользователя их может не быть.
- **Ярлык в меню** — файл `.desktop`: название (в том числе по-русски, `Name[ru]`), команда запуска `Exec`, значок `Icon`, категория меню.

`install.sh` построчно

```
#!/bin/sh                                        # «выполнять этот файл программой /bin/sh»
set -e                                           # остановиться на первой ошибке
HERE="$(cd "$(dirname "$0")" && pwd)"             # папка, где лежит сам скрипт
BIN="$HOME/.local/bin"                           # личные папки пользователя — sudo не нужен
APPS="$HOME/.local/share/applications"           # сюда кладутся ярлыки меню
ICONS="$HOME/.local/share/icons/hicolor/256x256/apps"
if [ "$1" = "--remove" ]; then ... fi            # ./install.sh --remove — удалить
mkdir -p "$BIN" "$APPS" "$ICONS"                 # создать папки (-p — вместе с родительскими, без
ошибки, если есть)
cp "$HERE/FabryPerot" "$BIN/FabryPerot"          # скопировать программу
chmod +x "$BIN/FabryPerot"                       # разрешить запуск (x — execute)
sed "s|^Exec=.*|Exec=$BIN/FabryPerot|" … > … # подставить в ярлык полный путь к программе
update-desktop-database "$APPS" 2>/dev/null || true   # обновить меню; ошибки не показывать
```

`$0` — путь к самому скрипту, `$1` — первый аргумент, `$HOME` — домашняя папка. `sed "s|что|на что|"` — замена по регулярному выражению: `^Exec=.*` — строка, начинающаяся с `Exec=`. `>` — записать вывод в файл, `2>/dev/null` — выбросить сообщения об ошибках. `tar -czf архив.tar.gz папка` — упаковать папку в сжатый архив (c — создать, z — сжать gzip, f — в файл).

11.6. Самопроверка `--selftest`

fabry_perot/app.py — main

```python
def main():
    selftest = "--selftest" in sys.argv
    app = create_app([arg for arg in sys.argv if arg != "--selftest"])
    window = MainWindow()
    window.show()
    if selftest:
        def finish():
            window.recalc()
            print("selftest ok" if window.results else "selftest failed", flush=True)
            app.exit(0 if window.results else 1)
        QtCore.QTimer.singleShot(1500, finish)
    sys.exit(app.exec())
```

Собранная программа запускается с ключом `--selftest`: открывает окно, через 1,5 с делает расчёт, печатает «selftest ok» и выходит с **кодом завершения** 0 (успех) или 1 (ошибка). Код завершения — число, которое программа возвращает системе; сборка проверяет его. Так ловятся ошибки, которых нет при запуске из исходников: забытая библиотека, не найденный модуль Qt. Ключ убирается из списка аргументов, прежде чем передать его Qt.

```
$p = Start-Process $exe -ArgumentList '--selftest' -Wait -PassThru
if ($p.ExitCode -ne 0) { throw "selftest failed" }
```

На Windows программа с окном (`--windowed`) не ждёт завершения в консоли, поэтому её запускают через `Start-Process -Wait -PassThru`: дождаться и вернуть объект процесса с кодом завершения; `-ne` — «не равно»; `throw` — ошибка, которая остановит сборку.

11.7. Значки

`tools/make_icon.py` рисует значок той же функцией `app_icon_image`, что и окно, в картинку 1024 × 1024 и сохраняет через библиотеку **Pillow** (работа с картинками) в три формата: PNG (Linux), ICO — несколько размеров от 16 до 256 в одном файле (Windows

выбирает нужный), ICNS — то же для macOS. Значки лежат в `assets/` готовыми, перерисовывать их нужно только при изменении рисунка.

11.8. Файлы requirements

Файл	Для чего
`requirements.txt`	запуск из исходников: `numpy>=1.19`, `PyQt6>=6.4` (любая версия не ниже)
`requirements/win7.txt`	сборка для Windows: закреплённые PyQt5 и numpy
`requirements/modern.txt`	сборка для macOS и Linux: закреплённые PyQt6 и numpy
`requirements/build.txt`	pytest и Pillow
`requirements/docs.txt`	сборка методичек

PyQt6 состоит из двух пакетов: `PyQt6` (переходник) и `PyQt6-Qt6` (сама библиотека Qt). Их версии закреплены отдельно. Строки с `#` — комментарии.

ЧАСТЬ 12. GIT И GITHUB ACTIONS

12.1. git

> **ПРОСТЫМИ СЛОВАМИ**
> git — программа, которая хранит всю историю изменений кода. Каждое сохранённое состояние называется **коммит**: «снимок» всех файлов с описанием и автором. Можно посмотреть любой прошлый снимок, сравнить, откатиться.

Понятие	Что это
репозиторий	папка проекта вместе со скрытой папкой `.git`, где хранится история
коммит	снимок с описанием; у каждого уникальный номер (хэш), например `72bf077`
ветка	отдельная линия развития: изменения в ней не мешают основной ветке (`main`), пока их не слили
remote, origin	копия репозитория на сервере (GitHub); `origin` — её имя по умолчанию
push / pull	отправить свои коммиты на сервер / забрать чужие
тег	постоянная метка на коммите, например `v2.0.0`
pull request	на GitHub — предложение слить ветку в основную, с обсуждением и проверками

```
git status                        # что изменено
git diff                          # сами изменения построчно
git add fabry_perot/window.py     # отметить файл для следующего коммита
git commit -m "Описание"          # сделать коммит
git push -u origin имя-ветки      # отправить ветку на GitHub
git log --oneline                 # история
git tag v2.0.0 && git push origin v2.0.0   # выпустить версию
```

`.gitignore` — список файлов, которые git не хранит (временные, собранные, `__pycache__` — папка, где Python кэширует скомпилированные модули).

12.2. GitHub Actions

GitHub бесплатно даёт на время виртуальные компьютеры с Windows, macOS и Linux. **GitHub Actions** по инструкции из файла `.github/workflows/build.yml` при каждом `push` берёт такие компьютеры, скачивает код, ставит библиотеки, запускает тесты и собирает программы. Результаты (**артефакты**) можно скачать со страницы запуска.

YAML

Файл написан на языке **YAML** — это способ записать словари и списки текстом:

```
ключ: значение               # словарь: ключ → значение
вложенный:                   # вложенность — отступом (пробелы, не табуляция!)
  внутренний: 1
список:
  - первый                   # элемент списка — с дефиса
  - второй
многострочный: |             # «|» — следующий блок текста как есть, со всеми строками
  команда 1
  команда 2
```

Разбор build.yml

Запись	Смысл
`name: Build`	имя, которое видно во вкладке Actions
`on: push: branches: ["**"], tags: ["v*"]`	когда запускать: при push в любую ветку и при теге, начинающемся с v; `workflow_dispatch` — ещё и кнопкой вручную
`permissions: contents: read`	права: только читать код (безопасность); задание release просит `write`, чтобы выложить файлы
`env:`	переменные окружения для всех шагов (`PYTHONIOENCODING: utf-8` — чтобы русские сообщения печатались на Windows)
`jobs:`	задания: test, windows, macos, linux, release. Каждое идёт на своём чистом компьютере
`runs-on: windows-2022`	на каком компьютере
`needs: test`	начать только после успешного задания test
`strategy: matrix: include: [...]`	запустить задание несколько раз с разными значениями (Python 3.8 + PyQt5 и 3.11 + PyQt6; два Мака)
`fail-fast: false`	если один вариант матрицы упал, остальные доводить до конца
`${{ matrix.python }}`	подстановка значения (выражение GitHub, вычисляется до запуска)
`steps:`	шаги задания по порядку
`uses: actions/checkout@v4`	готовое действие: скачать код репозитория (@v4 — версия действия)
`uses: actions/setup-python@v5, with: python-version, architecture: x86`	поставить Python нужной версии и разрядности
`run: …`	выполнить команды оболочки (bash на Linux и Mac, PowerShell с `shell: pwsh`)
`uses: actions/upload-artifact@v4`	сохранить файл как артефакт запуска
`if: startsWith(github.ref, 'refs/tags/v')`	выполнять задание только для тегов v*
`actions/download-artifact, merge-multiple`	собрать артефакты всех заданий в одну папку
`softprops/action-gh-release@v2`	действие стороннего автора: создать Release и приложить файлы; `generate_release_notes` — список изменений сам

Порядок: сначала **test** (два варианта), потом параллельно **windows**, **macos** (два), **linux**, в каждом снова тесты на своей системе, сборка, самопроверка, упаковка, артефакт; для тегов в конце **release**.

Как скачать готовую программу

1. На странице репозитория — вкладка **Actions**.

2. Слева **Build**, в списке — последний запуск с зелёной галочкой.
3. Внизу страницы запуска — **Artifacts**: щелчок по имени скачивает zip (GitHub всегда заворачивает артефакт в zip; внутри — exe, dmg или tar.gz). Артефакты хранятся 90 дней, скачивать может только вошедший на GitHub пользователь.
4. Для постоянной ссылки нужен выпуск: тег `v…` → файлы появляются в **Releases**, их может скачать кто угодно.

ЧАСТЬ 13. КАК СДЕЛАНЫ МЕТОДИЧКИ

Методички — не документы Word, а страницы HTML, которые программа `tools/build_docs.py` превращает в PDF. Это позволяет брать код, формулы, рисунки и снимки прямо из программы.

1. **Снимки окна**: `tools/docs_shots.py` запускает программу без экрана (offscreen), выставляет нужные параметры, делает `grab()` и сохраняет PNG. На снимок главного окна дорисовываются номера-выноски.
2. **Шаблоны** `docs/src/*.html` — обычный HTML (язык разметки веб-страниц: `<h1>` заголовок, `<p>` абзац, `<table>` таблица) плюс особые метки:
   - `<code ref="physics.airy"/>` — модуль **ast** разбирает `physics.py` как дерево, находит функцию `airy`, берёт её строки из файла; **pygments** раскрашивает код;
   - `<eq>…</eq>` и текст между знаками доллара — формулы в записи LaTeX (язык формул: `\frac{a}{b}` — дробь, `\sqrt{x}` — корень, `\lambda` — λ). **latex2mathml** переводит их в MathML — язык формул, который браузер рисует сам;
   - `<fig name="…"/>` — рисунок SVG из `tools/docs_figures.py`. SVG — векторная картинка, записанная текстом (линии, кривые, надписи), чёткая при любом увеличении. Графики на рисунках считаются функциями `physics.py`;
   - `<py name="…"/>` — таблица, посчитанная `tools/docs_tables.py` (ответы для 30 вариантов).
3. Метки заменяются **регулярными выражениями** (`re.sub`): шаблон вроде `<code\s[^>]*/>` находит все такие места, функция подставляет готовый HTML. Уже вставленные куски временно заменяются метками-заглушками, чтобы знак доллара в коде не приняли за формулу.
4. **PDF**: библиотека **Playwright** управляет браузером Chromium без окна: открывает страницу и печатает её в PDF формата A4 с полями и номерами страниц. Оформление (шрифты, отступы, как в методичках МГТУ) — в `style.css` (CSS — язык оформления веб-страниц).

```
pip install -r requirements/docs.txt
python tools/build_docs.py              # всё
python tools/build_docs.py guide        # только файлы, в имени которых есть «guide»
python tools/build_docs.py --no-shots   # не переснимать окно
```

ЧАСТЬ 14. ЕСЛИ ЧТО-ТО НЕ РАБОТАЕТ

`ModuleNotFoundError: No module named 'PyQt6'` (или numpy)

Библиотека не установлена в тот Python, которым запускаете. `python -m pip install -r requirements.txt` — именно через `python -m pip`, чтобы pip был того же Python.

Windows 7: PyQt6 не ставится или программа из исходников не запускается

Qt 6 не работает на Windows 7. Нужен Python 3.8 и `pip install -r requirements/win7.txt`. Или готовый exe.

Windows 7: «api-ms-win-crt-runtime-l1-1-0.dll не найдена»

Нет UCRT. В готовой сборке она внутри; если сборка сделана без неё — поставить обновление KB2999226.

Одиночный exe долго стартует

Он распаковывается при каждом запуске (часть 11.2). Используйте portable-версию.

Антивирус ругается на exe

Загрузчик PyInstaller иногда ложно принимают за вирус, потому что он распаковывает код во временную папку. Помогает portable-версия или добавление в исключения.

macOS: «программа повреждена и не может быть открыта»

`xattr -cr "/Applications/Fabry-Perot.app"` в Терминале (часть 11.4).

Linux: «could not load the Qt platform plugin "xcb"»

Не хватает системной библиотеки. `sudo apt install libxcb-cursor0 libxkbcommon-x11-0`. Запуск с `QT_DEBUG_PLUGINS=1 ./FabryPerot` покажет, какой именно.

Всё мелкое или мыльное на экране с масштабом

Проверьте масштаб в настройках экрана; программа сама подстраивается (часть 6.10). В Qt 5 можно задать `QT_SCALE_FACTOR=1.25`.

Хочу сбросить все настройки

Удалить файл `FabryPerot.ini` (часть 6.8). Программа начнёт с первого примера.

Лабораторная «застряла» после перезапуска

Так задумано: незавершённая работа продолжается. Кнопка «Завершить» выключает режим.

Тесты падают с ошибкой про дисплей

Задайте `QT_QPA_PLATFORM=offscreen`. Для проверки старой версии Qt — `FABRY_QT=5`.

Сборка на GitHub красная

Actions → запуск → задание с красным крестиком → шаг с ошибкой: там полный текст. Чаще всего это упавший тест: его имя и значения видны в выводе pytest.

ЧАСТЬ 15. СЛОВАРЬ ТЕРМИНОВ

Антиалиасинг (сглаживание)
полупрозрачные пиксели по краю линии, чтобы край не был «лесенкой».

Артефакт
файл, который сохранился после запуска GitHub Actions (собранная программа).

Бисекция
поиск значения делением отрезка пополам (часть 4.8).

Векторизация
запись вычислений над целыми массивами numpy вместо циклов.

Виджет
элемент окна Qt.

Гамма-коррекция
возведение яркости в степень 1/2,2, чтобы монитор показал её правильно.

Декоратор
строка `@имя` перед функцией, изменяющая её поведение (`@property`, `@pytest.fixture`).

Доверительная граница
полуширина интервала, в который истинное значение попадает с вероятностью 0,95.

Замыкание
внутренняя функция, запомнившая переменные внешней.

Зерно (seed)
число или строка, задающие последовательность псевдослучайных чисел.

Интерполяция
оценка значения между известными точками (линейная — по прямой).

Кэш
сохранённый готовый результат, чтобы не считать заново.

Кодировка
правило перевода символов в байты (UTF-8).

Код завершения
число, которое программа возвращает системе при выходе: 0 — успех.

Компоновка (layout)
объект Qt, расставляющий виджеты.

Контраст
Tmax/Tmin = 1 + F.

Коэффициент Стьюдента
множитель для доверительной границы при малом числе измерений.

Кольца равного наклона
кольца, каждое из которых соответствует одному углу θ.

МНК

метод наименьших квадратов: прямая с наименьшей суммой квадратов отклонений.

Муар

ложный узор из-за того, что мелкая структура не совпадает с сеткой пикселей.

Область свободной дисперсии Δλ (FSR)

расстояние между соседними пиками пропускания.

Перекрытие порядков

когда пик одной линии ложится на соседний пик другой ($\delta\lambda \geq \Delta\lambda$).

Первообразная

функция, производная которой равна данной; разность её значений — интеграл.

Порядок интерференции m

сколько длин волн укладывается в $2nd \cdot \cos\theta'$.

Премультиплированный цвет

каналы, заранее умноженные на прозрачность.

Растеризация

перевод фигуры в пиксели.

Разрешающая способность

λ/w — во сколько раз длина волны больше наименьшей различимой разницы.

Регулярное выражение

шаблон для поиска текста (`^Exec=.*`).

Резкость (finesse)

$\Delta\lambda/w$ — во сколько раз пик уже расстояния между пиками.

Сигнал и слот

механизм Qt «событие → вызов подключённой функции».

Свёртка

замена каждого значения взвешенной суммой соседей.

Селектор

часть правила QSS, указывающая, к каким виджетам оно относится.

Степени свободы

число измерений минус число найденных из них параметров.

Тег

постоянная метка на коммите git (номер версии).

Функция Эйри

$T = T_{max}/(1 + F \sin^2(\delta/2))$ — пропускание интерферометра.

Ширина пика w (FWHM)

ширина на половине высоты.

Broadcasting

«растягивание» массивов numpy разной формы до общей.

Bootloader (загрузчик)

маленькая программа PyInstaller, которая запускает встроенный Python.

CI
: continuous integration — автоматическая проверка и сборка при каждом изменении (у нас GitHub Actions).

CSV
: таблица в виде текста с разделителями.

dtype
: тип чисел массива numpy.

Event loop
: цикл событий Qt.

Fixture
: подготовка данных для тестов pytest.

HiDPI
: экраны, где в логической точке больше одного физического пикселя.

JSON
: текстовый формат словарей и списков.

Offscreen
: режим Qt без экрана.

Portable
: программа без установки, запускаемая из папки.

QSS
: язык оформления виджетов Qt.

Tween
: плавный переход значения от старого к новому.

UCRT
: системная библиотека C от Microsoft, нужная Python.

YAML
: текстовый формат настроек (build.yml).